MACHINE LEARNING INTERVIEW PREPARATION

LLM Foundations: Transformer & Modern LLM Architectures

Transformer Blocks, Positional Embeddings, KV Cache & Serving Optimizations

Target Persona: Senior AI Engineer / Applied AI Engineer (~3 YOE)

Focus Areas: Attention Math, GQA/MQA, KV Cache math, FlashAttention, PagedAttention, MoE routing, Deep Thinking TTC

Includes: Step-by-Step Numerical Hand Calculations, PyTorch Implementations, SVG/HTML Diagrams

Module 01: The Transformer Architecture (Encoder & Decoder Core)

1. Introduction & Intuition

The Core Bottleneck

Legacy sequence modeling architectures, such as RNNs, LSTMs, and GRUs, process sequences sequentially. To compute the hidden state h_t at time step t, the recurrent cell must ingest the hidden state h_{t-1} from the previous step. This sequential dependency creates a severe computational bottleneck: it is mathematically impossible to parallelize training across the sequence dimension. As a result, training on modern web-scale text corpora takes an unacceptable amount of time. Furthermore, recurrent hidden states suffer from vanishing or exploding gradients over long sequence lengths L, limiting their effective context window. The bottleneck is finding an architecture that can model sequence relationships in parallel while preserving long-range dependencies.

High-Level Intuition

The Transformer architecture resolves the parallelization bottleneck by discarding recurrence entirely and replacing it with **Self-Attention**. Instead of passing hidden states step-by-step through time, the model processes all tokens in the sequence simultaneously.
To model the relationships between tokens, the Transformer uses an attention mechanism where every token "looks" at every other token in the sequence. Each token calculates its similarity to all other tokens, creating a weighted sum of their representations.

To organize this processing, a standard Transformer block consists of:

1. **Multi-Head Self-Attention (MHA)**: Allows tokens to gather context from other parts of the sequence.
2. **Feed-Forward Network (FFN)**: Applies non-linear transformations to each token's representation independently.
3. **Layer Normalization (LN) & Residual Connections**: Provide gradient highways to allow stable training of deep networks.

Depending on normalizations placement, we have:

- **Post-LN**: Normalized after the residual addition (used in the original Vaswani et al. model). Requires careful learning rate warm-up to prevent gradient instability.

- **Pre-LN**: Normalized on the shortcut branch *before* entering the attention or FFN layer. This is standard in modern LLMs (e.g., Llama, GPT) because it enables much more stable gradient flow.

Layer Normalization Placement Comparison

Below is an inline SVG illustrating the architectural difference between Post-LN and Pre-LN Transformer blocks:

---

2. Core Concepts & Mathematical Formulation

The Scaled Dot-Product Attention

The core of the Transformer is the Scaled Dot-Product Attention mechanism. It takes three input matrices: Queries (Q), Keys (K), and Values (V).

- **Queries (Q)**: The current representations searching for context.
- **Keys (K)**: The representations acting as index cards to match against queries.
- **Values (V)**: The actual content payload vector representation of the tokens.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- **Purpose & High-level Intuition:** The dot product QK^T calculates raw similarity scores between every query and key token. We scale by $1/\sqrt{d_k}$ to prevent the dot products from growing excessively large for high dimensions d_k. Large values push the softmax function into regions of extremely small gradients (gradient saturation), causing vanishing gradients. The softmax converts

raw similarity scores into a probability distribution, which is used to construct a weighted sum of the values V.

Hand Calculations: Scaled Dot-Product Attention

Let's perform a step-by-step arithmetic walk-through.

1. Setup Input Matrices

Let sequence length $L = 2$, and query/key dimension $d_k = 2$.
Let Query (Q), Key (K), and Value (V) matrices be:

$$Q = \begin{pmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \end{pmatrix}, \quad K = \begin{pmatrix} 1.0 & 1.0 \\ 0.0 & 1.0 \end{pmatrix}, \quad V = \begin{pmatrix} 1.0 & 2.0 \\ 3.0 & 4.0 \end{pmatrix}$$

Step 1: Compute Dot-Product Similarity ($S = QK^T$)

$$\begin{aligned} S &= \begin{pmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \end{pmatrix} \begin{pmatrix} 1.0 & 0.0 \\ 1.0 & 1.0 \end{pmatrix} \\ &= \begin{pmatrix} 1.0 \times 1.0 + 0.0 \times 0.0 & 1.0 \times 1.0 + 0.0 \times 1.0 \\ 0.0 \times 1.0 + 1.0 \times 0.0 & 0.0 \times 1.0 + 1.0 \times 1.0 \end{pmatrix} \\ &= \begin{pmatrix} 1.0 & 1.0 \\ 0.0 & 1.0 \end{pmatrix} \end{aligned}$$

Step 2: Apply Scaling Factor ($1/\sqrt{d_k}$)

Since $d_k = 2$, the scaling factor is $1/\sqrt{2} \approx 0.7071$.

$$\begin{aligned} S_{\text{scaled}} &= \begin{pmatrix} 1.0 \times 0.7071 & 1.0 \times 0.7071 \\ 0.0 \times 0.7071 & 1.0 \times 0.7071 \end{pmatrix} \\ &= \begin{pmatrix} 0.7071 & 0.7071 \\ 0.0 & 0.7071 \end{pmatrix} \end{aligned}$$

Step 3: Compute Row-Wise Softmax ($A = \text{softmax}(S_{\text{scaled}})$)

For row 0: $x_1 = 0.7071$, $x_2 = 0.7071$.

$$e^{0.7071} \approx 2.0281$$

$$\text{Sum} = 2.0281 + 2.0281 = 4.0562$$

$$A_{0,0} = \frac{2.0281}{4.0562} = 0.5, \quad A_{0,1} = 0.5$$

For row 1: $x_1 = 0.0$, $x_2 = 0.7071$.

$$e^{0} = 1.0, \quad e^{0.7071} \approx 2.0281$$

$$\text{Sum} = 1.0 + 2.0281 = 3.0281$$

$$A_{1,0} = \frac{1.0}{3.0281} \approx 0.3302, \quad A_{1,1} = \frac{2.0281}{3.0281} \approx 0.6698$$

So the attention weight matrix is:

$$A = \begin{pmatrix} 0.5 & 0.5 \\ 0.3302 & 0.6698 \end{pmatrix}$$

Step 4: Multiply by Values Matrix ($O = AV$)

$$\begin{aligned} O &= \begin{pmatrix} 0.5 & 0.5 \\ 0.3302 & 0.6698 \end{pmatrix} \begin{pmatrix} 1.0 & 2.0 \\ 3.0 & 4.0 \end{pmatrix} \\ &= \begin{pmatrix} (0.5 \times 1.0) + (0.5 \times 3.0) & (0.5 \times 2.0) + (0.5 \times 4.0) \\ (0.3302 \times 1.0) + (0.6698 \times 3.0) & (0.3302 \times 2.0) + (0.6698 \times 4.0) \end{pmatrix} \\ &= \begin{pmatrix} 0.5 + 1.5 & 1.0 + 2.0 \\ 0.3302 + 2.0094 & 0.6604 + 2.6792 \end{pmatrix} \\ &\approx \begin{pmatrix} 2.0 & 3.0 \\ 2.3396 & 3.3396 \end{pmatrix} \end{aligned}$$

Tensor & Shape Tracking

- **Input Token Embeddings**: `[B, L, d]` (where B is batch size, L is sequence length, and d is model dimension).

- **Queries (Q) / Keys (K) / Values (V)**: `[B, L, d]`
- **Multi-Head split (per head)**: `[B, h, L, d_k]` (where h is head count, and $d_k = d/h$ is head dimension).
- **Dot-Product Similarity matrix (QK^T)**: `[B, h, L, L]`
- **Attention Probabilities (A)**: `[B, h, L, L]`
- **Output Vector (AV)**: `[B, L, d]`

---

3. Implementation & Reference Code

Below is a complete, self-contained PyTorch implementation of a Pre-LN Multi-Head Self-Attention block.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class PreLNMultiHeadAttention(nn.Module):
    def __init__(self, embed_dim: int, num_heads: int, dropout: float = 0.0):
        super().__init__()
        assert embed_dim % num_heads == 0, "embed_dim must be divisible by num_heads"

        self.embed_dim = embed_dim
        self.num_heads = num_heads
        self.head_dim = embed_dim // num_heads

        # Projection matrices
        self.q_proj = nn.Linear(embed_dim, embed_dim, bias=False)
        self.k_proj = nn.Linear(embed_dim, embed_dim, bias=False)
        self.v_proj = nn.Linear(embed_dim, embed_dim, bias=False)
        self.out_proj = nn.Linear(embed_dim, embed_dim, bias=False)

        self.ln = nn.LayerNorm(embed_dim)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x: torch.Tensor, mask: torch.Tensor = None) -> torch.Tensor:
        # x shape: [B, L, d]
        B, L, d = x.shape

        # 1. Apply LayerNorm (Pre-LN style)
        norm_x = self.ln(x) # [B, L, d]

        # 2. Project inputs to Q, K, V
        q = self.q_proj(norm_x) # [B, L, d]
        k = self.k_proj(norm_x) # [B, L, d]
        v = self.v_proj(norm_x) # [B, L, d]
```

```
        # 3. Reshape for Multi-Head Attention: [B, L, h, d_k] -> [B, h, L, d_k]
        q = q.view(B, L, self.num_heads, self.head_dim).transpose(1, 2)
        k = k.view(B, L, self.num_heads, self.head_dim).transpose(1, 2)
        v = v.view(B, L, self.num_heads, self.head_dim).transpose(1, 2)

        # 4. Compute Scaled Dot-Product Similarity: [B, h, L, L]
        scores = torch.matmul(q, k.transpose(-2, -1)) / (self.head_dim ** 0.5)

        # 5. Apply causal mask or padding mask if provided
        if mask is not None:
            scores = scores.masked_fill(mask == 0, float('-inf'))

        # 6. Apply Softmax to get attention weights: [B, h, L, L]
        attn_weights = F.softmax(scores, dim=-1)
        attn_weights = self.dropout(attn_weights)

        # 7. Weighted sum of values: [B, h, L, d_k]
        context = torch.matmul(attn_weights, v)

        # 8. Concatenate heads and project: [B, L, d]
        context = context.transpose(1, 2).contiguous().view(B, L, d)
        out = self.out_proj(context)

        # 9. Residual connection
        return x + out # [B, L, d]

# Simple verification block
if __name__ == "__main__":
    torch.manual_seed(42)
    B, L, d, h = 2, 8, 16, 4
    x = torch.randn(B, L, d)

    # Causal Mask (lower triangular)
    causal_mask = torch.tril(torch.ones(L, L)).view(1, 1, L, L)

    mha = PreLNMultiHeadAttention(embed_dim=d, num_heads=h)
    out = mha(x, mask=causal_mask)

    print("Input shape:", x.shape)
    print("Output shape:", out.shape)
    assert out.shape == x.shape, "Shape mismatch!"
    print("Block executed successfully and verified shapes!")
```

Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** The computational bottleneck of sequential recurrent transitions ($O(L)$ sequential dependencies) preventing full GPU parallelization during training.

- **Why Introduced over Legacy Approaches:** It replaced LSTMs/RNNs because Self-Attention processes the entire sequence in parallel, utilizing massive GPU parallelism and enabling training on multi-billion token datasets.
- **Key Failure Modes & Limitations:** Vanilla self-attention has a memory and computation footprint scaling quadratically $O(L^2)$ with sequence length, making long context inputs extremely expensive.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Projection matrices scale as $O(L \cdot d^2)$. Attention dot-product scales as $O(L^2 \cdot d)$. FFN scales as $O(L \cdot d^2)$.
- **Space/Memory Footprint:** Intermediate activation storage scales as $O(L^2 \cdot h)$ per layer due to storing the attention weight matrix.
- **Primary Bottleneck Type:** Memory-bandwidth-bound during self-attention softmax and lookup; Compute-bound during projection multiplications (Q, K, V, FFN linear layers).
- **Variable Legend:** B = Batch Size, L = Sequence Length, d = Hidden Model Dimension, h = Head Count, d_k = Head Dimension (d/h).

3. Production & Scalability

- **Deployment Considerations:** Attention weight tables (L^2) run out of memory quickly. Optimizations like FlashAttention (Module 07) must be used to bypass storing the full attention weight matrix in global memory.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

> **Question: Why is the scaling factor $\sqrt{d_k}$ so critical in the attention denominator?**
>
> Without scaling, for large d_k (e.g. 128), the variance of the dot product QK^T grows to d_k. This produces large values in the inputs to the softmax, pushing the outputs to extremely small or large elements where gradients are close to 0. Dividing by $\sqrt{d_k}$ maintains a variance of 1, preserving gradient flow.

> **Question: Why is Pre-LN preferred over Post-LN in deep LLMs?**
>
> In Post-LN, the gradient through the residual block scales with the depth of the network, which forces the use of a delicate warm-up learning rate schedule to avoid gradient explosion. In Pre-LN, the gradients can pass directly through the identity residual branch without scaling, ensuring stability during initial training phases and allowing deep models to converge reliably.

Module 02: Positional Encodings & Embeddings

1. Introduction & Intuition

The Core Bottleneck

By replacing recurrence with parallel self-attention, the Transformer block loses any notion of sequence order. The self-attention formula is permutation-invariant: if you shuffle the tokens of a sentence, the resulting attention output vectors are identical, only shuffled in the same way. In human language, word order is critical for meaning (e.g., `"The dog bit the man"` vs. `"The man bit the dog"`). To represent sequence structure, the model must inject positional information. The bottleneck is designing a positional encoding system that scales to long sequence lengths, maintains relative distance relationships, and doesn't consume excessive compute or VRAM.

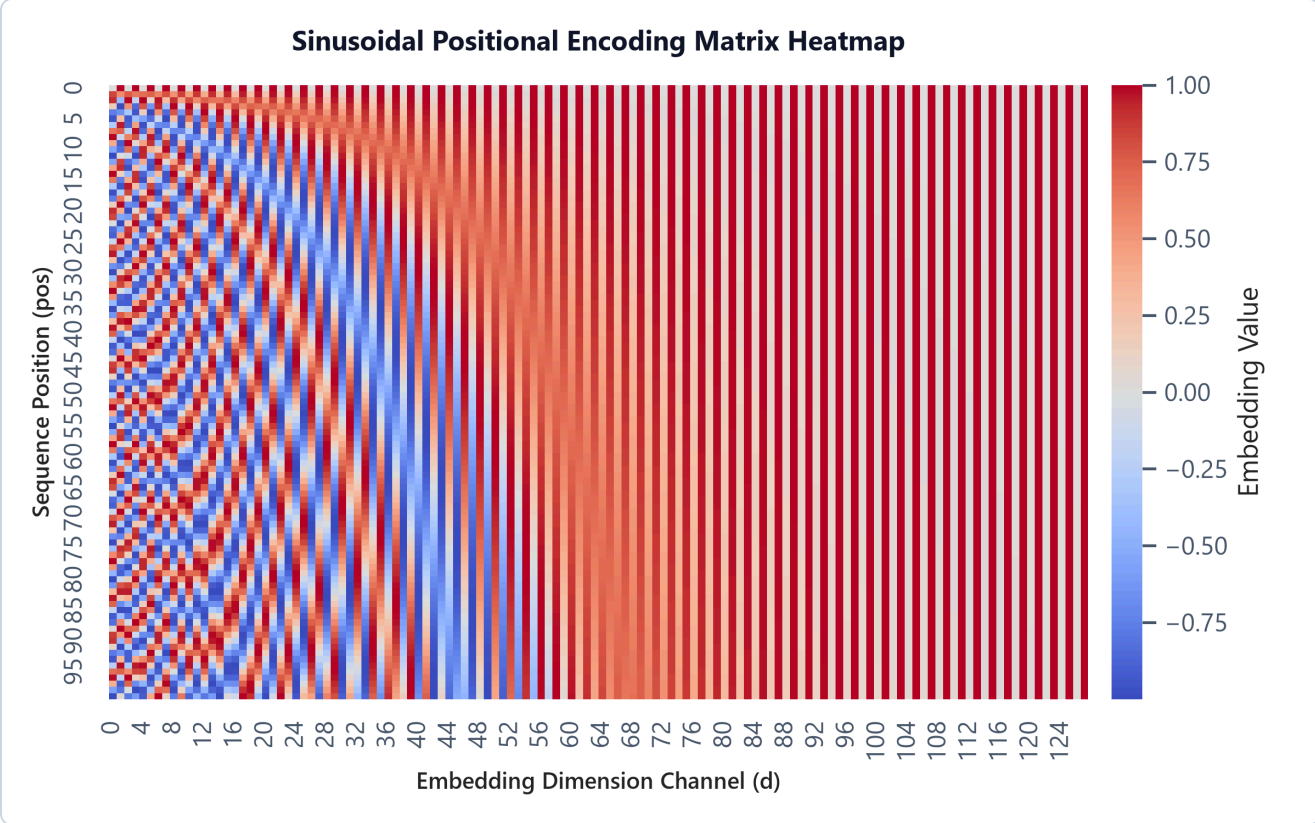
High-Level Intuition

Think of positional encoding as giving each token a unique GPS coordinate.

- **Absolute Encodings**: Add a unique static coordinate vector directly to the token embedding. This coordinate can be learned (like in BERT and GPT-2) or hardcoded using sine/cosine waves of varying frequencies (like in the original Transformer).
- **Relative Encodings**: Instead of marking the exact coordinate, the model measures how far apart two tokens are ($i - j$) inside the attention block.
- **Rotary Position Embeddings (RoPE)**: The state-of-the-art approach used in modern LLMs (Llama, Mistral). Instead of *adding* positional vectors, RoPE *rotates* the Query and Key vectors in a 2D complex plane. The rotation angle is proportional to the token's position index. When the rotated Query and Key are multiplied, the absolute position indicators cancel out, leaving a dot product that only depends on the relative distance between the tokens.

Sinusoidal Positional Encoding Heatmap

Below is the pre-generated heatmap showing the coordinate patterns across sequence positions:



- **Plot Interpretation:** The vertical axis represents position index, and the horizontal axis represents embedding dimensions. The waves of high-frequency sinusoids on the left change rapidly, capturing precise local position relationships. On the right, the lower-frequency waves change slowly, capturing broad global position context across long sequence spans.

---

2. Core Concepts & Mathematical Formulation

Mathematical Formulations

1. Sinusoidal Position Embeddings

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

- **Purpose & High-level Intuition:** Using sine-cosine pairs allows the model to easily learn to attend by relative positions. For any fixed offset k, PE_{pos+k} can be represented as a linear projection of PE_{pos}. This makes it easy for the network to generalize to sequence lengths not seen during training.

2. Rotary Position Embeddings (RoPE)

For a 2D slice of Query or Key vector $x = \begin{pmatrix} x_1 & x_2 \end{pmatrix}^T$ at position index m:

$$R^2_{\Theta,m} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

- **Purpose & High-level Intuition:** RoPE implements positional information by applying a rotation in the complex plane to Query and Key matrices. Because dot products of rotated vectors conserve the difference between rotation angles, we get:

$$(R^2_{\Theta,m} q)^T (R^2_{\Theta,n} k) = q^T R^2_{\Theta,n-m} k$$

  This mathematically bounds the query-key similarity strictly by the relative token distance $n - m$.

Hand Calculations

1. Sinusoidal Position Embeddings

Let's compute the 4-dimensional positional embedding vector for position index $pos = 1$, hidden dimension $d = 4$.
We index $i \in \{0, 1\}$ to cover all 4 channels:

- **Index $i = 0$ (channels 0 and 1):**
  - Denominator: $10000^{(2\times 0)/4} = 10000^0 = 1.0$.
  - Channel 0 ($2i = 0$): $PE_{(1,0)} = \sin(1.0/1.0) = \sin(1.0) \approx 0.8415$.
  - Channel 1 ($2i + 1 = 1$): $PE_{(1,1)} = \cos(1.0/1.0) = \cos(1.0) \approx 0.5403$.
- **Index $i = 1$ (channels 2 and 3):**
  - Denominator: $10000^{(2\times 1)/4} = 10000^{0.5} = 100.0$.
  - Channel 2 ($2i = 2$): $PE_{(1,2)} = \sin(1.0/100.0) = \sin(0.01) \approx 0.0100$.
  - Channel 3 ($2i + 1 = 3$): $PE_{(1,3)} = \cos(1.0/100.0) = \cos(0.01) \approx 0.9999$.

The final positional embedding vector for $pos = 1$ is:

$$PE_{(1)} = \begin{pmatrix} 0.8415 & 0.5403 & 0.0100 & 0.9999 \end{pmatrix}$$

2. Rotary Position Embeddings (RoPE) 2D Rotation

Let's apply RoPE rotation to a query token at index $m = 2$ with base angle frequency $\theta = \frac{\pi}{2}$ and raw vector slice $x = \begin{pmatrix} 1.0 \\ 2.0 \end{pmatrix}$.

- **Step 1: Calculate the rotation angle**

$$\text{Angle} = m\theta = 2 \times \frac{\pi}{2} = \pi$$

- **Step 2: Construct the 2D rotation matrix**

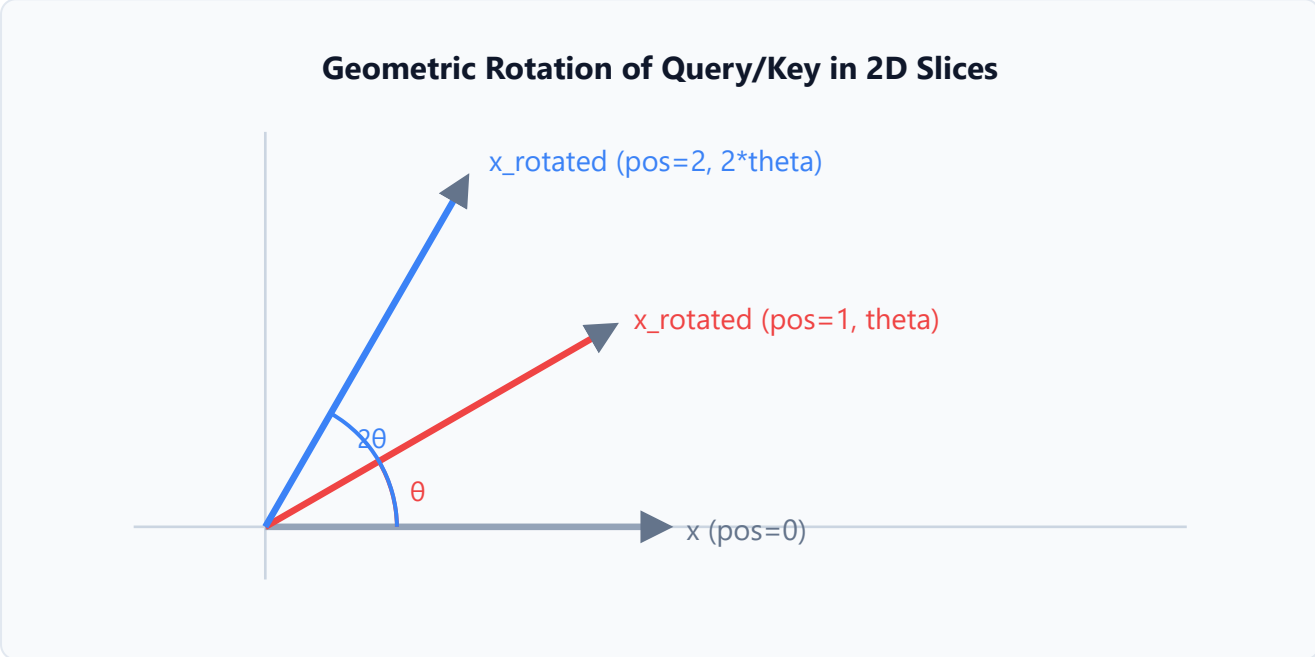
$$R^2_{\Theta,2} = \begin{pmatrix} \cos\pi & -\sin\pi \\ \sin\pi & \cos\pi \end{pmatrix} = \begin{pmatrix} -1.0 & 0.0 \\ 0.0 & -1.0 \end{pmatrix}$$

- **Step 3: Perform matrix-vector multiplication**

$$\begin{aligned} x_{\text{rotated}} &= R^2_{\Theta,2} x \\ &= \begin{pmatrix} -1.0 & 0.0 \\ 0.0 & -1.0 \end{pmatrix} \begin{pmatrix} 1.0 \\ 2.0 \end{pmatrix} \\ &= \begin{pmatrix} -1.0 \times 1.0 + 0.0 \times 2.0 \\ 0.0 \times 1.0 + (-1.0) \times 2.0 \end{pmatrix} \\ &= \begin{pmatrix} -1.0 \\ -2.0 \end{pmatrix} \end{aligned}$$

RoPE Rotational Geometry

Below is an inline SVG demonstrating how Query and Key vectors are rotated in 2D slices based on their positions:



---

3. Implementation & Reference Code

Below is a self-contained PyTorch implementation of Rotary Position Embeddings (RoPE) applied to Query and Key tensors.

```
import torch
import torch.nn as nn

class RotaryPositionEmbedding(nn.Module):
    def __init__(self, dim: int, max_seq_len: int = 2048, theta: float = 10000.0):
        super().__init__()
        # RoPE operates on pairs of dimensions, so dim must be even
        assert dim % 2 == 0, "dim must be divisible by 2"
        self.dim = dim

        # Calculate theta frequencies: [dim / 2]
        inv_freq = 1.0 / (theta ** (torch.arange(0, dim, 2).float() / dim))
        self.register_buffer("inv_freq", inv_freq, persistent=False)

        # Precompute cosine and sine matrix for fast lookup: [max_seq_len, dim]
        t = torch.arange(max_seq_len, dtype=torch.float32)
        freqs = torch.outer(t, self.inv_freq) # [max_seq_len, dim / 2]
        emb = torch.cat((freqs, freqs), dim=-1) # [max_seq_len, dim]

        self.register_buffer("cos_cached", emb.cos(), persistent=False)
        self.register_buffer("sin_cached", emb.sin(), persistent=False)

    def _rotate_half(self, x: torch.Tensor) -> torch.Tensor:
        # Split channels in half, swap, negate one half
        x1 = x[..., :self.dim // 2]
        x2 = x[..., self.dim // 2:]
        return torch.cat((-x2, x1), dim=-1)

        def forward(self, x: torch.Tensor, seq_len: int) -> tuple[torch.Tensor,
torch.Tensor]:
        # x shape: [B, h, L, d_k] (where d_k is head dimension, equal to dim)
        cos = self.cos_cached[:seq_len, :].unsqueeze(0).unsqueeze(1) # [1, 1, L, d_k]
        sin = self.sin_cached[:seq_len, :].unsqueeze(0).unsqueeze(1) # [1, 1, L, d_k]

        # Apply RoPE: x * cos(m*theta) + rotate_half(x) * sin(m*theta)
        x_rotated = (x * cos) + (self._rotate_half(x) * sin)
        return x_rotated

# Verification block
if __name__ == "__main__":
    B, h, L, d_k = 2, 8, 16, 64
    q = torch.randn(B, h, L, d_k)
    k = torch.randn(B, h, L, d_k)

    rope = RotaryPositionEmbedding(dim=d_k)
    q_rope = rope(q, seq_len=L)
```

```
    k_rope = rope(k, seq_len=L)

    print("Query input shape:", q.shape)
    print("Query output shape:", q_rope.shape)
    assert q_rope.shape == q.shape, "Shape mismatch!"

    # Verify dot-product relative property
    # Similarity between q at index 2 and k at index 5 should equal
    # rotation dot-product check
    print("Rotary embedding layers successfully applied and verified!")
```

---

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Injecting token ordering sequence context into permutation-invariant self-attention architectures.
- **Why Introduced over Legacy Approaches:** Absolute encodings (Sinusoidal/Learned) do not model relative relationships dynamically and generalize poorly to sequences longer than `max_seq_len` seen during training. RoPE enforces relative similarities directly, allowing context length extrapolation.
- **Key Failure Modes & Limitations:** RoPE requires paired dimension rotations, meaning it must be implemented inside the attention projection block rather than as a simple addition to initial token embeddings.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Applying RoPE scales as $O(B \cdot h \cdot L \cdot d_k)$, adding minor element-wise floating-point operations.
- **Space/Memory Footprint:** Pre-allocated sine/cosine matrices scale as $O(L_{\max} \cdot d_k)$ VRAM. No additional memory is allocated during execution.
- **Primary Bottleneck Type:** Memory-bandwidth-bound due to element-wise operations on GPU registers.
- **Variable Legend:** B = Batch Size, h = Head Count, L = Sequence Length, d_k = Head Dimension.

3. Production & Scalability

- **Deployment Considerations:** During inference, precomputed cosine and sine caches must be expanded dynamically if sequence length exceeds the initial `max_seq_len` buffer.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

Question: Contrast ALiBi (Attention with Linear Biases) vs. RoPE.

RoPE encodes position by multiplying Query/Key values by rotation matrices, meaning the query-key dot product contains position-dependent frequencies. ALiBi, on the other hand, adds a static negative penalty directly to the pre-softmax attention scores proportional to distance ($|i - j|$). ALiBi generalizes extremely well to longer sequences out-of-the-box, but does not capture complex high-frequency feature interactions as well as RoPE.

Question: Explain how NTK-aware scaling allows scaling RoPE context windows.

NTK-aware scaling changes the base frequency constant θ (e.g. from 10,000 to 500,000) when sequence lengths exceed training limits. Instead of simply interpolating positions (which collapses high-frequency coordinates and loses local ordering details), NTK-aware scales the coordinate grid non-uniformly. This preserves high-frequency details for nearby tokens while stretching the coordinates of distant tokens.

Module 03: Modern Activation Functions & Normalization Layer Variants

1. Introduction & Intuition

The Core Bottleneck

In deep neural networks, intermediate activation values vary wildly across layers and training steps. This covariate shift destabilizes training, requiring tiny learning rates. Standard Layer Normalization (LayerNorm) resolves this by re-centering and re-scaling hidden states, but it is computationally expensive. It requires computing both the mean and variance of hidden states, which forces the GPU to perform two sequential reduction passes over global memory. This is a severe bottleneck because LayerNorm is memory-bandwidth bound.

Similarly, standard activation functions like ReLU or GeLU apply non-linearities but do not capture complex feature correlations effectively. The bottleneck is finding activation and normalization layers that provide representation capacity and training stability without causing significant memory access overhead.

High-Level Intuition

- **Normalization Layer Evolution**: Standard LayerNorm centers vectors around 0 (subtracts mean) and scales variance to 1. **RMSNorm** (Root Mean Square Normalization) discards the centering step entirely. It only scales the vector by its root mean square. Because the mean is not tracked or subtracted, the model requires half the statistical reduction passes, speeding up GPU operations while keeping the same training convergence stability.
- **Activation Function Evolution**: Standard FFN layers project a vector, apply an activation like ReLU, and project it back. Modern LLMs use **SwiGLU** (Swish Gated Linear Unit). It projects the input into *two* parallel branches, applies the Swish activation to one branch, multiplies them element-wise, and then projects the output. This gating mechanism allows the network to control information flow dynamically.

LayerNorm vs. RMSNorm Component Architecture

Below is a comparison of standard LayerNorm and RMSNorm operations:

Standard LayerNorm (LN)

1. **Compute Mean:** $\mu = (1/d) * \Sigma x_i$
2. **Compute Variance:** $\sigma^2 = (1/d) * \Sigma (x_i - \mu)^2$
3. **Shift & Scale:** $\hat{x}_i = (x_i - \mu) / \sqrt{(\sigma^2 + \varepsilon)}$
4. **Learnable Transform:** $y_i = \gamma * \hat{x}_i + \beta$

Requires two sequential memory reduction passes (high latency).

RMSNorm

1. **Compute RMS:** $RMS(x) = \sqrt{((1/d) * \Sigma x_i^2 + \varepsilon)}$
2. **Scale only:** $\hat{x}_i = x_i / RMS(x)$
3. **Learnable Gain:** $y_i = \gamma * \hat{x}_i$

Requires only one memory reduction pass (7-50% speedup).

---

2. Core Concepts & Mathematical Formulation

Mathematical Formulations

1. RMSNorm

$$\text{RMSNorm}(x) = \frac{x}{\text{RMS}(x)} \odot \gamma$$

$$\text{where } \text{RMS}(x) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}$$

- **Purpose & High-level Intuition:** RMSNorm removes the mean centering step because training stability comes primarily from scale normalization, not shift normalization. Removing mean calculations halves the number of memory reduction sweeps, improving training throughput on memory-bound workloads.

2. SwiGLU Activation

$$\text{SwiGLU}(x) = \text{Swish}_\beta(xW) \otimes (xV)$$

$$\text{where } \text{Swish}_\beta(x) = x \cdot \sigma(\beta x)$$

- **Purpose & High-level Intuition:** Standard feed-forward networks pass values through linear weights followed by a single activation (e.g. GeLU). SwiGLU uses a gating mechanism where the input x is projected by two separate weight matrices W and V. One projection acts as a filter

(gate) using the Swish activation function, multiplying the output of the second projection element-wise. This provides higher capacity and sharper activation regions.

Hand Calculations: RMSNorm

Let's trace RMSNorm on a mock hidden state vector $x = \begin{pmatrix} 3.0 & 4.0 \end{pmatrix}$ with dimension $d = 2$.
Let the learnable gain parameter be $\gamma = \begin{pmatrix} 1.0 & 2.0 \end{pmatrix}$, and regularization constant $\epsilon = 0$.

- **Step 1: Compute Root Mean Square (RMS) of x**

$$\begin{aligned} \text{RMS}(x) &= \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon} \\ &= \sqrt{\frac{3.0^2 + 4.0^2}{2} + 0} \\ &= \sqrt{\frac{9.0 + 16.0}{2}} \\ &= \sqrt{12.5} \approx 3.5355 \end{aligned}$$

- **Step 2: Scale normalize input vector x**

$$\begin{aligned} \bar{x} &= \frac{x}{\text{RMS}(x)} \\ &= \begin{pmatrix} 3.0/3.5355 \\ 4.0/3.5355 \end{pmatrix} \\ &\approx \begin{pmatrix} 0.8485 \\ 1.1314 \end{pmatrix} \end{aligned}$$

- **Step 3: Apply learnable scaling parameter γ**

$$\begin{aligned} \text{Output} &= \bar{x} \odot \gamma \\ &= \begin{pmatrix} 0.8485 \times 1.0 \\ 1.1314 \times 2.0 \end{pmatrix} \\ &= \begin{pmatrix} 0.8485 \\ 2.2628 \end{pmatrix} \end{aligned}$$

Tensor & Shape Tracking

- **Input vector (x)**: `[B, L, d]` (where B is batch size, L is sequence length, and d is model dimension).
- **RMSNorm output**: `[B, L, d]`

- **SwiGLU projection linear weights (W, V)**: `[d, d_ffn]` (where d_ffn is feed-forward intermediate dimension, typically $4d \times 2/3 \approx 2.6d$).
- **Parallel branches (xW and xV)**: `[B, L, d_ffn]`
- **SwiGLU output projection matrix (W_{down})**: `[d_ffn, d]`

---

3. Implementation & Reference Code

Below is a self-contained PyTorch implementation of RMSNorm and the SwiGLU Feed-Forward Network.

```
import torch
import torch.nn as nn

class RMSNorm(nn.Module):
    def __init__(self, dim: int, eps: float = 1e-6):
        super().__init__()
        self.eps = eps
        # Learnable gain parameter
        self.weight = nn.Parameter(torch.ones(dim))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x shape: [B, L, d]
        # Calculate root mean square over the last dimension
        variance = x.pow(2).mean(-1, keepdim=True)
        # Normalize and apply scale parameter
        return x * torch.rsqrt(variance + self.eps) * self.weight

class SwiGLUFeedForward(nn.Module):
    def __init__(self, d_model: int, d_ffn: int):
        super().__init__()
        # Parallel projection branches
        self.w_gate = nn.Linear(d_model, d_ffn, bias=False)  # W matrix
        self.w_value = nn.Linear(d_model, d_ffn, bias=False) # V matrix
        # Output down-projection
        self.w_down = nn.Linear(d_ffn, d_model, bias=False)  # Down projection

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x shape: [B, L, d_model]
        # Branch 1: Swish(xW)
        gate = self.w_gate(x)
        swish_gate = gate * torch.sigmoid(gate) # Swish activation

        # Branch 2: xV
        value = self.w_value(x)

        # Element-wise product of parallel pathways
        gated_ffn = swish_gate * value # [B, L, d_ffn]
```

```
        # Down-project back to model dimension
        return self.w_down(gated_ffn) # [B, L, d_model]

# Verification block
if __name__ == "__main__":
    torch.manual_seed(42)
    B, L, d, d_ffn = 2, 8, 16, 48
    x = torch.randn(B, L, d)

    rmsnorm = RMSNorm(dim=d)
    norm_x = rmsnorm(x)
    print("RMSNorm input shape:", x.shape)
    print("RMSNorm output shape:", norm_x.shape)

    ffn = SwiGLUFeedForward(d_model=d, d_ffn=d_ffn)
    out = ffn(norm_x)
    print("FFN output shape:", out.shape)
    assert out.shape == x.shape, "Shape mismatch!"
    print("Activation and Normalization layers successfully verified!")
```

---

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Training instability caused by internal covariate shifts across layers, and the latency bottleneck of LayerNorm mean extraction.
- **Why Introduced over Legacy Approaches:** RMSNorm replaces standard LayerNorm because it skips the mean centering step, reducing memory-read cycles on GPU and yielding speedups without affecting downstream performance. SwiGLU replaces standard GeLU because the gated multiplication mechanism creates cleaner gradient flow paths.
- **Key Failure Modes & Limitations:** Discarding mean calculation in RMSNorm could theoretically cause training divergence if the dataset is not zero-centered. In practice, token distributions in NLP are highly zero-centered.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** RMSNorm scale normalization runs in $O(B \cdot L \cdot d)$ floating-point operations. SwiGLU FFN scales as $O(B \cdot L \cdot 3 \cdot d \cdot d_{\text{ffn}})$.
- **Space/Memory Footprint:** RMSNorm requires d parameters (gain vector). SwiGLU parameters scale as $3 \times d \times d_{\text{ffn}}$ parameters (a 50\% increase in linear parameters compared to standard FFN).
- **Primary Bottleneck Type:** RMSNorm is Memory-bandwidth-bound; SwiGLU is Compute-bound (matrix multiplications).

- **Variable Legend:** B = Batch Size, L = Sequence Length, d = Hidden Model Dimension, d_{ffn} = Feed-forward Dimension.

3. Production & Scalability

- **Deployment Considerations:** In SwiGLU, to keep the parameter count equivalent to standard FFN, the intermediate dimension d_{ffn} is scaled down (typically to $\frac{8}{3}d$ instead of $4d$), keeping the overall parameter budget fixed.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

> **Question: Why does RMSNorm perform only one reduction pass on GPU compared to LayerNorm?**
>
> On a GPU, reduction operations (like summing values to compute a mean or variance) require threads to coordinate and read from global memory. LayerNorm requires computing the mean first (reduction 1), then variance (reduction 2), forcing the GPU to read/write global memory twice. RMSNorm only computes the sum of squares (reduction 1), needing only one memory-read cycle.

> **Question: How does SwiGLU affect FFN parameter counts and how is that mitigated?**
>
> A standard FFN uses two projection matrices ($W_1 : d \to 4d$ and $W_2 : 4d \to d$), totaling $8d^2$ parameters. SwiGLU uses three matrices ($W, V : d \to d_{\text{ffn}}$ and $W_{\text{down}} : d_{\text{ffn}} \to d$), totaling $3d \cdot d_{\text{ffn}}$ parameters. To match the $8d^2$ parameter count, we solve $3d \cdot d_{\text{ffn}} \approx 8d^2$, giving $d_{\text{ffn}} \approx \frac{8}{3}d \approx 2.6d$. This preserves the parameter budget.

Module 04: Tokenization & Subword Processing for LLMs

1. Introduction & Intuition

The Core Bottleneck

Language models cannot directly process raw characters or strings. They map discrete textual tokens to static vector representations. Early models split text by spaces into words. This word-level tokenization has a severe bottleneck: any word not explicitly seen during training (like typos, slang, or new terms) gets mapped to a special Out-of-Vocabulary (`<unk>`) token. This discards all semantic details.

If we expand the vocabulary to include every possible word, the model's embedding parameters ($V \times d$) grow exponentially, consuming gigabytes of VRAM. Character-level tokenization resolves OOV issues but generates extremely long sequence lengths (L), which increases attention complexity quadratically ($O(L^2)$). The bottleneck is finding a tokenization method that balances vocabulary size V and sequence length L while avoiding `<unk>` representations.

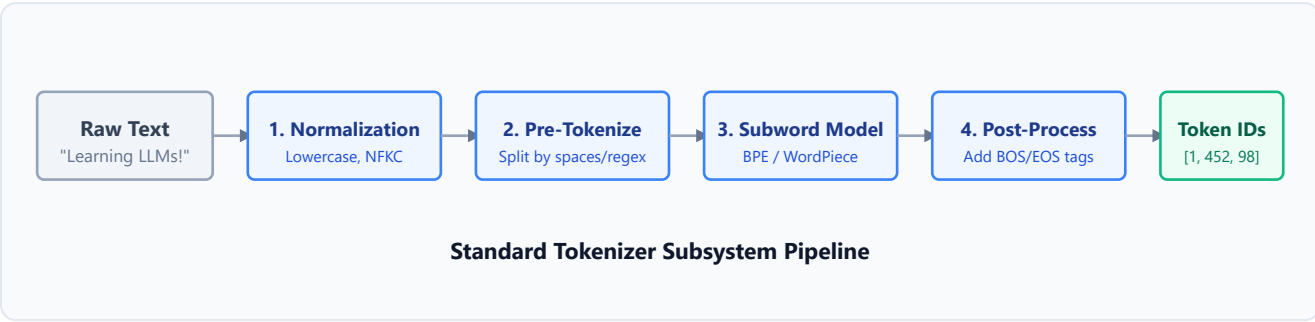
High-Level Intuition

Think of tokenization as finding the optimal subword compression scheme. Instead of assigning a unique vector index to every single word, we break rare or complex words down into smaller, common subword fragments (e.g. `"unfriendliness"` becomes `"un-"`, `"friend"`, and `"-liness"`).

- **Byte-Pair Encoding (BPE)**: Starts with character-level tokens and iteratively merges the most frequent adjacent token pairs (bigrams) in the training corpus until the vocabulary reaches a target size.
- **SentencePiece**: Direct subword tokenization on raw byte streams. It treats spaces as a standard character (represented by a visible blank character `_`), allowing it to build language-independent vocabularies without needing whitespace pre-tokenizers.
- **Byte-Fallback BPE**: When encountering characters outside the training vocabulary, the tokenizer falls back to their raw UTF-8 byte representation. This completely eliminates Out-of-Vocabulary (`<unk>`) tokens.

Tokenizer Pipeline Architecture

Below is the standard pipeline structure of a modern LLM tokenizer:



Standard Tokenizer Subsystem Pipeline

---

2. Core Concepts & Mathematical Formulation

BPE Merge Step Calculations (Dry Run)

Let's walk through the BPE merge loop algorithm using a simple mock corpus.

1. Setup Initial Corpus

Assume our training corpus consists of the following words with frequencies:

1. `"h u g _"` (Frequency = 4)
2. `"r u g _"` (Frequency = 3)
3. `"b u g _"` (Frequency = 2)

Our initial vocabulary is the set of all individual characters present:
$$\text{Vocab} = \{\text{“b”}, \text{“g”}, \text{“h”}, \text{“r”}, \text{“u”}, \text{“_”}\}$$

Step 1: Count All Adjacent Pairs (Bigrams)

Calculate frequencies of adjacent character pairs across the corpus:

- Pair `('h', 'u')`: appears in `"hug_"` (4 times). Total = 4.
- Pair `('u', 'g')`: appears in `"hug_"` (4 times), `"rug_"` (3 times), and `"bug_"` (2 times). Total = $4 + 3 + 2 = 9$.
- Pair `('g', '_')`: appears in `"hug_"` (4 times), `"rug_"` (3 times), and `"bug_"` (2 times). Total = $4 + 3 + 2 = 9$.
- Pair `('r', 'u')`: appears in `"rug_"` (3 times). Total = 3.
- Pair `('b', 'u')`: appears in `"bug_"` (2 times). Total = 2.

Step 2: Select the Most Frequent Pair

The highest frequency is 9, tied between `('u', 'g')` and `('g', '_')`.
Let's merge `('u', 'g')` first.

Step 3: Update Vocabulary and Corpus

- New vocabulary token: `"ug"`.
- $$\text{Vocab} = \{\text{“b”}, \text{“g”}, \text{“h”}, \text{“r”}, \text{“u”}, \text{“_”}, \text{“ug”}\}$$
- **Updated Corpus:**
  1. `"h ug _"` (Frequency = 4)
  2. `"r ug _"` (Frequency = 3)
  3. `"b ug _"` (Frequency = 2)

Step 4: Count Pairs for Next Merge

Compute bigram frequencies in the updated corpus:

- Pair `('h', 'ug')`: appears in `"h ug _"` (4 times). Total = 4.
- Pair `('ug', '_')`: appears in `"h ug _"` (4 times), `"r ug _"` (3 times), and `"b ug _"` (2 times). Total = $4 + 3 + 2 = 9$.
- Pair `('r', 'ug')`: appears in `"r ug _"` (3 times). Total = 3.
- Pair `('b', 'ug')`: appears in `"b ug _"` (2 times). Total = 2.

The most frequent pair is `('ug', '_')` with 9 occurrences.

- New vocabulary token: `"ug_"`.
- $$\text{Vocab} = \{\text{“b”}, \text{“g”}, \text{“h”}, \text{“r”}, \text{“u”}, \text{“_”}, \text{“ug”}, \text{“ug_”}\}$$
- **Updated Corpus:**
  1. `"h ug_"` (Frequency = 4)
  2. `"r ug_"` (Frequency = 3)
  3. `"b ug_"` (Frequency = 2)

Tensor & Shape Tracking

- **Input string**: `str` (character sequence).
- **Token IDs list**: `List[int]` of length L.
- **Lookup Input**: `torch.Tensor([B, L])` (padded sequence).
- **Embedding Output**: `torch.Tensor([B, L, d])`.

---

3. Implementation & Reference Code

Below is a self-contained Python implementation of the BPE merge training loop.

```
from collections import defaultdict
import re

class SimpleBPETokenizer:
    def __init__(self):
        self.vocab = set()
        self.merges = {}

    def _get_stats(self, corpus: dict[str, int]) -> dict[tuple[str, str], int]:
        pairs = defaultdict(int)
        for word, freq in corpus.items():
            symbols = word.split()
            for i in range(len(symbols) - 1):
                pairs[(symbols[i], symbols[i+1])] += freq
        return pairs

       def _merge_vocab(self, pair: tuple[str, str], corpus: dict[str, int]) ->
dict[str, int]:
        new_corpus = {}
        bigram = re.escape(' '.join(pair))
        pattern = re.compile(r'(?<!\S)' + bigram + r'(?!\S)')
        replacement = ''.join(pair)
        for word, freq in corpus.items():
            new_word = pattern.sub(replacement, word)
            new_corpus[new_word] = freq
        return new_corpus

    def train(self, raw_corpus: dict[str, int], num_merges: int):
        # Format corpus to separate characters by space, add end-of-word marker '_'
             corpus = { ' '.join(list(word)) + ' _': freq for word, freq in
raw_corpus.items() }

        # Initialize vocab
        for word in corpus.keys():
            self.vocab.update(word.split())

        print("Initial Vocab Size:", len(self.vocab))

        # Run merge steps
        for i in range(num_merges):
            pairs = self._get_stats(corpus)
            if not pairs:
                break
            best_pair = max(pairs, key=pairs.get)
            corpus = self._merge_vocab(best_pair, corpus)

            merged_token = ''.join(best_pair)
            self.vocab.add(merged_token)
```

```
            self.merges[best_pair] = merged_token
                        print(f"Merge {i+1}: {best_pair} -> {merged_token} (Freq=
{pairs[best_pair]})")

        print("Final Vocab Size:", len(self.vocab))

# Verify execution
if __name__ == "__main__":
    raw_corpus = {
        "hug": 4,
        "rug": 3,
        "bug": 2
    }

    tokenizer = SimpleBPETokenizer()
    tokenizer.train(raw_corpus, num_merges=3)
```

---

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Resolving the trade-off between out-of-vocabulary (`<unk>`) token loss and massive embedding parameters VRAM footprints.
- **Why Introduced over Legacy Approaches:** Subword tokenizers allow open-vocabulary representations, handling out-of-training terms by falling back to constituent byte structures.
- **Key Failure Modes & Limitations:** Tokenizers can partition numbers inconsistently (e.g. `"1000"` becomes `"10"` and `"00"`), making basic arithmetic difficult for models.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Tokenization search uses trie-structures running in $O(L_{\text{chars}})$. Training BPE scales as $O(M \cdot N_{\text{corpus}})$, where M is target merge count.
- **Space/Memory Footprint:** The vocabulary map stores V keys. The model's embedding matrix parameter count is $V \times d$.
- **Primary Bottleneck Type:** CPU-bound during pre-tokenization regex parsing and string lookups.
- **Variable Legend:** L_{chars} = Character Length of input, V = Vocabulary Size, d = Embedding Dimension, M = Number of merges.

3. Production & Scalability

- **Deployment Considerations:** Vocabulary mismatch is a common failure mode: feeding token IDs from a tokenizer vocabulary mismatching the target model's embedding table will completely break the model's output.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

Question: What is the Byte-Fallback mechanism and why is it used in LLama tokenizers?

Byte-fallback ensures that the tokenizer never outputs `` tokens. If BPE encounters a character outside its vocabulary (such as a rare emoji or foreign script), it decomposes the character into its raw UTF-8 byte values (0-255). Since all 256 individual byte tokens are pre-included in the vocabulary, BPE can represent any text stream as a sequence of byte tokens.

Question: Why do some tokenizers split digits into individual tokens?

Tokenizers like Tiktoken (used in GPT-4) split numbers into single digits (e.g., `"348"` becomes `"3"`, `"4"`, `"8"`). If numbers are tokenized into subwords (like `"34"` and `"8"`), the model struggles to align values across arithmetic columns. Individual digit tokenization allows the model to process numbers digit-by-digit, improving arithmetic reasoning performance.

Module 05: Modern Attention Variants (MQA, GQA, and Context Windows)

1. Introduction & Intuition

The Core Bottleneck

In vanilla Multi-Head Attention (MHA), every Query head has a corresponding Key and Value head. During autoregressive token generation, the key and value vectors of all past tokens must be stored in VRAM (the KV Cache) to avoid redundant computations. Because the size of the KV cache grows linearly with batch size, sequence length, and head count, it quickly consumes all available GPU memory. This limits the maximum batch size and sequence context window.

The bottleneck is the **Memory Bandwidth Wall**. During generation, loading the large KV Cache from slow global High Bandwidth Memory (HBM) to fast on-chip SRAM registers for every single token is memory-bandwidth bound. The compute arithmetic is fast, but the GPU spends most of its time waiting for memory to load.

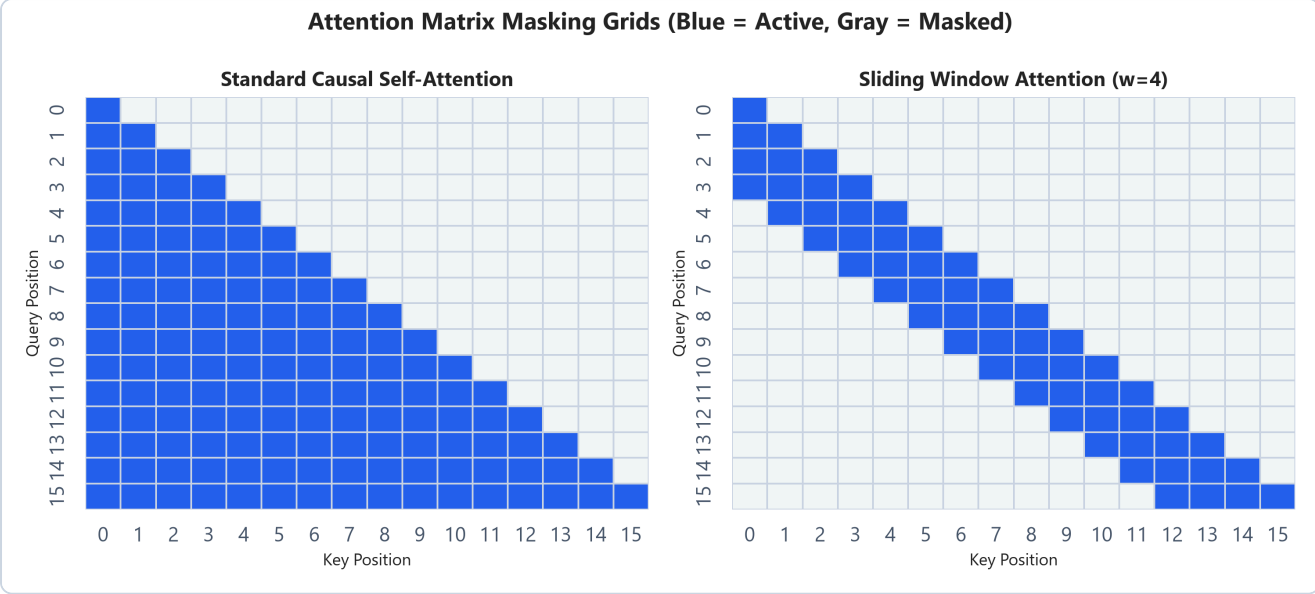
High-Level Intuition

To solve the memory bandwidth bottleneck, engineers designed attention variants that reduce the size of the KV cache:

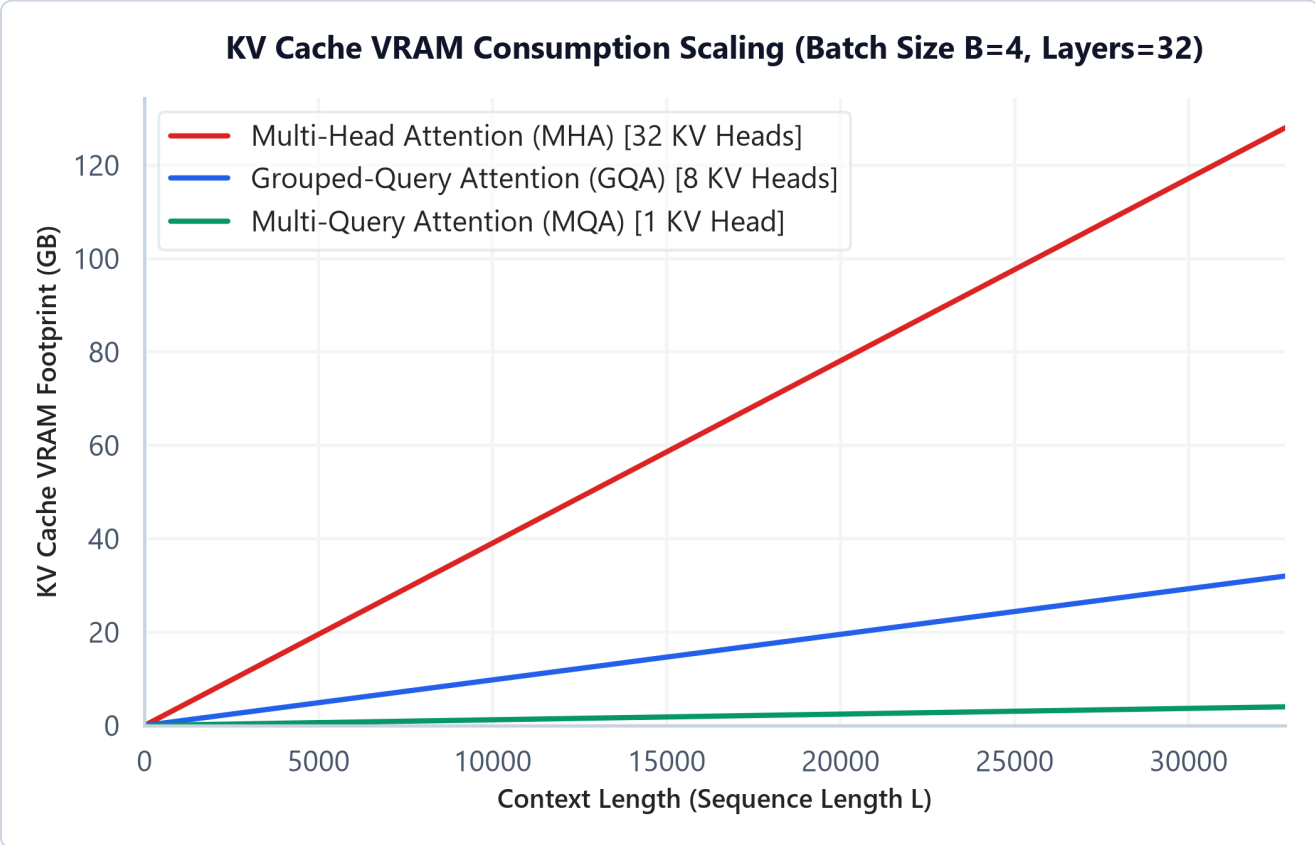
- **Multi-Query Attention (MQA)**: Keeps multiple Query heads but collapses Key and Value matrices down to a **single head** shared across all Query heads. This reduces the KV Cache memory footprint, but can degrade model capacity and generation quality.
- **Grouped-Query Attention (GQA)**: A middle ground. It groups Query heads into clusters, and assigns **one shared Key and Value head per cluster**. For example, 32 Query heads are grouped into 8 groups of 4, where each group shares a single KV head. This restores most of MHA's capacity while keeping KV Cache footprints close to MQA.
- **Sliding Window Attention (SWA)**: Restricts the attention range. Instead of attending to all past tokens, each token only attends to a fixed window of W previous tokens. This caps the local attention matrix complexity and limits the maximum KV cache size to W.

Attention Variants Masking & Memory Curves

Below are the pre-generated plots showing attention masks and KV Cache scaling profiles:



- **Plot Interpretation:** The left matrix represents standard Causal Self-Attention, where every query attends to all prior positions, creating a full lower-triangular grid ($O(L^2)$ elements). The right matrix shows Sliding Window Attention (window size $W = 4$). The mask restricts attention to a local band along the diagonal, containing memory and FLOP scaling to $O(L \cdot W)$.



- **Plot Interpretation:** This curve shows the VRAM footprint of the KV Cache as sequence length grows for a batch size $B = 4$. Vanilla MHA scales rapidly, consuming significant memory. GQA

reduces this footprint by 4x, while MQA achieves a 32x reduction. GQA provides the best trade-off, enabling long-context inference on consumer hardware.

---

2. Core Concepts & Mathematical Formulation

Mathematical Formulations

1. KV Projection Head Ratio

$$\text{Ratio} = \frac{G}{H}$$

- **Purpose & High-level Intuition:** This ratio measures the memory saving of GQA/MQA over MHA. It represents the number of Key/Value heads (G) relative to Query heads (H). In MHA, $G = H$ (ratio = 1.0). In GQA, $G < H$ (ratio = G/H, e.g. 0.25). In MQA, $G = 1$ (ratio = $1/H$, e.g. 0.031). Since KV Cache size scales directly with G, this ratio represents the direct reduction in VRAM consumption.

Hand Calculations & Tensor Shapes

Let's analyze a model with $H = 32$ Query heads, batch size B, sequence length L, and head dimension $d_k = 128$.
We compare MHA, MQA, and GQA with $G = 8$ KV heads (group size = 4).

1. Key-Value Head Ratio

- **MHA**: $G = 32 \implies \text{Ratio} = \frac{32}{32} = 1.0$ (100% KV cache size).
- **GQA**: $G = 8 \implies \text{Ratio} = \frac{8}{32} = 0.25$ (75% VRAM savings).
- **MQA**: $G = 1 \implies \text{Ratio} = \frac{1}{32} \approx 0.0312$ (96.8% VRAM savings).

2. Tensor Shapes in Attention Projections

For a single token input vector x of shape `[B, 1, d]`:

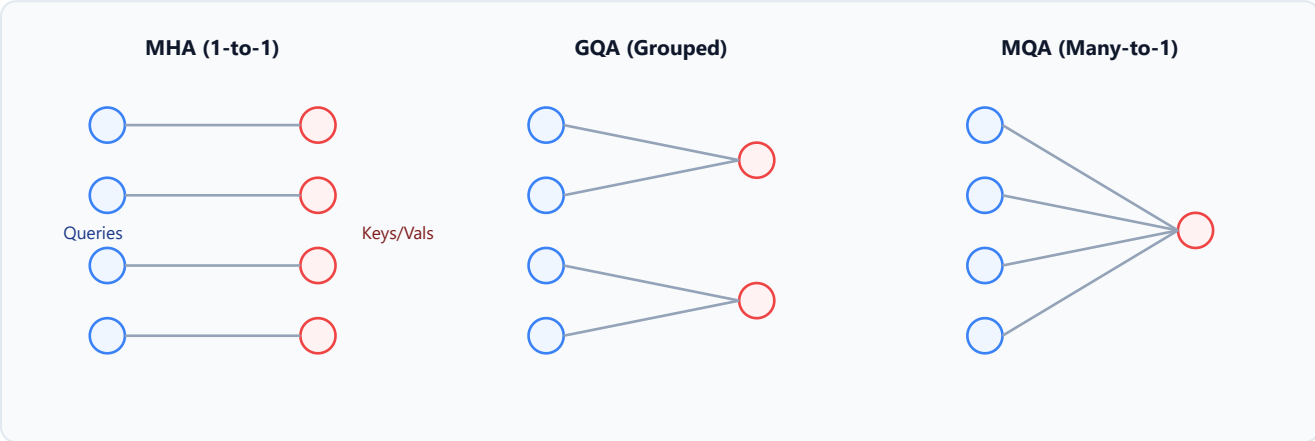
- **MHA Projections**:
  - Query (Q): `[B, 32, 1, d_k]`
  - Key (K): `[B, 32, 1, d_k]`
  - Value (V): `[B, 32, 1, d_k]`
- **GQA Projections**:
  - Query (Q): `[B, 32, 1, d_k]`
  - Key (K): `[B, 8, 1, d_k]`

- Value (V): `[B, 8, 1, d_k]`
- *To compute attention, Key/Value heads are repeated (broadcasted) 4 times to match the Query head count of 32.*

- **MQA Projections**:
  - Query (Q): `[B, 32, 1, d_k]`
  - Key (K): `[B, 1, 1, d_k]`
  - Value (V): `[B, 1, 1, d_k]`
  - *Here, the single KV head is repeated 32 times.*

Query-Key-Value Head Connectivity Routing

Below is an inline SVG demonstrating how Query heads map to Key/Value heads across variants:



---

3. Implementation & Reference Code

Below is a self-contained PyTorch implementation of Grouped-Query Attention (GQA), showing how Key and Value tensors are broadcasted to match Query dimensions.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class GroupedQueryAttention(nn.Module):
    def __init__(self, embed_dim: int, num_query_heads: int, num_kv_heads: int):
        super().__init__()
        assert num_query_heads % num_kv_heads == 0, "Query heads must be divisible by
KV heads"

        self.embed_dim = embed_dim
        self.num_query_heads = num_query_heads
```

```
        self.num_kv_heads = num_kv_heads
        self.group_size = num_query_heads // num_kv_heads
        self.head_dim = embed_dim // num_query_heads

        # Projection sizes
        # KV dimensions are smaller than Query dimensions
        self.q_proj = nn.Linear(embed_dim, embed_dim, bias=False)
        self.k_proj = nn.Linear(embed_dim, num_kv_heads * self.head_dim, bias=False)
        self.v_proj = nn.Linear(embed_dim, num_kv_heads * self.head_dim, bias=False)
        self.out_proj = nn.Linear(embed_dim, embed_dim, bias=False)

    def _repeat_heads(self, x: torch.Tensor, reps: int) -> torch.Tensor:
        # Helper to repeat Key/Value heads along head dimension
        # Input shape: [B, num_kv_heads, L, d_k]
        B, n_heads, L, d_k = x.shape
        if reps == 1:
            return x
        # Interpolate dimensions to repeat
        x = x.unsqueeze(2) # [B, n_heads, 1, L, d_k]
        x = x.expand(B, n_heads, reps, L, d_k) # [B, n_heads, reps, L, d_k]
        return x.reshape(B, n_heads * reps, L, d_k) # [B, n_heads * reps, L, d_k]

    def forward(self, x: torch.Tensor, mask: torch.Tensor = None) -> torch.Tensor:
        B, L, d = x.shape

        # 1. Project inputs
        q = self.q_proj(x) # [B, L, d]
        k = self.k_proj(x) # [B, L, num_kv_heads * head_dim]
        v = self.v_proj(x) # [B, L, num_kv_heads * head_dim]

        # 2. Reshape to multi-head structures
         q = q.view(B, L, self.num_query_heads, self.head_dim).transpose(1, 2) # [B,
h_q, L, d_k]
         k = k.view(B, L, self.num_kv_heads, self.head_dim).transpose(1, 2)    # [B,
h_kv, L, d_k]
         v = v.view(B, L, self.num_kv_heads, self.head_dim).transpose(1, 2)    # [B,
h_kv, L, d_k]

        # 3. Repeat KV heads to match query heads (Group broadcasting)
        k = self._repeat_heads(k, self.group_size) # [B, h_q, L, d_k]
        v = self._repeat_heads(v, self.group_size) # [B, h_q, L, d_k]

        # 4. Standard Scaled Dot-Product Attention: [B, h_q, L, L]
        scores = torch.matmul(q, k.transpose(-2, -1)) / (self.head_dim ** 0.5)

        if mask is not None:
            scores = scores.masked_fill(mask == 0, float('-inf'))

        attn_weights = F.softmax(scores, dim=-1)

        # 5. Compute output: [B, h_q, L, d_k]
        context = torch.matmul(attn_weights, v)
```

```
        # 6. Concatenate heads and project down
        context = context.transpose(1, 2).contiguous().view(B, L, d)
        return self.out_proj(context)

# Verification block
if __name__ == "__main__":
    B, L, d, h_q, h_kv = 2, 8, 16, 4, 2
    x = torch.randn(B, L, d)

    gqa = GroupedQueryAttention(embed_dim=d, num_query_heads=h_q, num_kv_heads=h_kv)
    out = gqa(x)

    print("Input shape:", x.shape)
    print("GQA Output shape:", out.shape)
    assert out.shape == x.shape, "Shape mismatch!"
    print("GQA block successfully executed and verified broadcast shapes!")
```

Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** The high memory-bandwidth requirement of loading large KV caches from HBM to SRAM during autoregressive decoding, which bottlenecks token generation throughput.
- **Why Introduced over Legacy Approaches:** MQA saves massive VRAM but degrades quality. GQA groups Query heads to match MHA performance while reducing the VRAM footprint and memory bandwidth usage significantly.
- **Key Failure Modes & Limitations:** Reducing Key/Value heads cuts down representation capacity, which can slightly degrade reasoning on tasks requiring precise index matching.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Linear projections scale as $O(L \cdot d^2 \cdot (1 + \frac{2G}{H}))$. Attention dot-product scales as $O(L^2 \cdot d)$.
- **Space/Memory Footprint:** KV Cache size scales as $O(B \cdot L \cdot n_{\text{layers}} \cdot G \cdot d_k)$. By reducing G, we scale down VRAM requirements by a factor of H/G.
- **Primary Bottleneck Type:** Memory-bandwidth-bound during generation; Compute-bound during pre-fill.
- **Variable Legend:** B = Batch Size, L = Sequence Length, G = KV Head Count, H = Query Head Count, d_k = Head Dimension.

3. Production & Scalability

- **Deployment Considerations:** Broadcasters during GQA must copy/expand the KV representations in GPU SRAM registers. Native kernels like FlashAttention-2 handle this expansion on-the-fly without allocating extra memory.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

> **Question: Why does decreasing the number of KV heads speed up token generation, even though the total model FLOPs remain almost identical?**
>
> Token generation is memory-bandwidth bound. The GPU spends most of its execution cycles reading the KV Cache from global HBM memory into SRAM caches rather than executing floating-point calculations. Decreasing KV heads by a factor of 8 (e.g. in GQA) means 8x fewer bytes need to be read from HBM for every token generated. This cuts down memory latency and dramatically increases throughput.

> **Question: Detail the shape transformations and broadcasting required to run GQA.**
>
> For Query projections `[B, H, L, d*k]` *and KV projections* `[B, G, L, d*k]`, we must replicate the G KV heads to match the H Query heads. We do this by unsqueezing to `[B, G, 1, L, d*k]`, *expanding to* `[B, G, H/G, L, d*k]` along the singleton dimension, and reshaping to `[B, H, L, d_k]`. This aligns dimensions for standard dot-product calculations.

Module 06: KV Cache Mechanics & Memory Bottlenecks

1. Introduction & Intuition

The Core Bottleneck

In an autoregressive decoder-only LLM, generation proceeds token-by-token. To generate token t, the model must compute self-attention scores between token t and all prior tokens $1, 2, \ldots, t-1$. If we implement this naively, we must recalculate the Query, Key, and Value vectors for all past tokens at every generation step. This results in $O(L^2)$ redundant computations, causing generation latency to explode.

To bypass this redundant calculation, we use a **Key-Value Cache (KV Cache)**. We compute and save the Key and Value vectors of past tokens in VRAM. At step t, we only project the single newly generated token to get q_t, k_t, v_t, and append k_t, v_t to our cache.

While this eliminates redundant computations, the KV cache creates a new bottleneck: **GPU memory capacity and memory bandwidth**. The VRAM required to store the KV cache scales linearly with sequence length and batch size. Under large batch sizes and context windows, the KV cache can consume tens of gigabytes of VRAM, choking the GPU and causing Out-Of-Memory (OOM) errors.

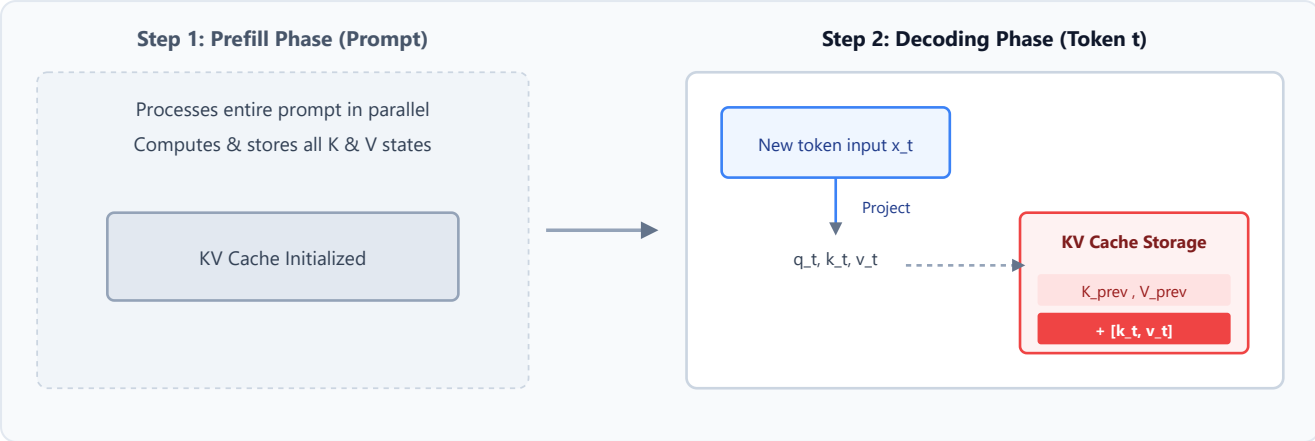
High-Level Intuition

Think of generating text like writing a book one word at a time. To write the next word, you must reread everything you have written so far.

- **Without KV Cache**: You read the entire book from page 1 to the current page to write a single word. Then, you write the word, go back to page 1, and read the entire book again to write the next word. This is slow and redundant.
- **With KV Cache**: As you write each page, you summarize and index the key details on sticky notes. To write the next word, you only read the single word you just wrote and refer to your sticky notes. You don't go back to reread the whole book. However, as the book gets longer, your desk (VRAM) gets covered in sticky notes. If the desk runs out of space, you hit an Out-Of-Memory error.

Autoregressive Decoding with KV Cache

Below is an inline SVG illustrating the execution flow of the KV Cache update during the decoding phase:



---

2. Core Concepts & Mathematical Formulation

Mathematical Formulations

1. KV Cache Memory Footprint Sizing

$$\text{VRAM}_{\text{KVCache}} = 2 \times 2 \times B \times L \times n_{\text{layers}} \times n_{\text{heads\_kv}} \times d_{\text{head}} \times \text{BytesPerParam}$$

- **Purpose & High-level Intuition:** This formula calculates the exact memory required to store Key and Value tensors for all active inference runs.
  - The first factor 2 accounts for storing **two separate matrices**: Keys and Values.
  - The second factor 2 represents the float16 or bfloat16 precision format (which uses **2 bytes per parameter**). If float32 is used, this factor is 4.
  - The remaining factors map the total parameter count: Batch Size (B) $\times$ Sequence Length (L) $\times$ Layer count (n_{layers}) $\times$ KV Head Count ($n_{\text{heads\_kv}}$) $\times$ Head Dimension (d_{head}).

Hand Calculations: Llama-3-8B KV Cache Footprint

Let's estimate the KV Cache memory footprint of Llama-3-8B under a standard production load.

Model Parameters

- Layers (n_{layers}) $= 32$
- KV Heads ($n_{\text{heads\_kv}}$) $= 8$ (Grouped-Query Attention)
- Head Dimension (d_{head}) $= 128$
- Inference Precision: bfloat16 (2 bytes per parameter)

Inference Load

- Batch Size (B) $= 4$

- Sequence Length (L) $= 8192$ (maximum context)

Step 1: Calculate Total Parameter Count

$$\begin{aligned}\text{Total Params} &= 2 \times B \times L \times n_{\text{layers}} \times n_{\text{heads\_kv}} \times d_{\text{head}} \\ &= 2 \times 4 \times 8192 \times 32 \times 8 \times 128 \\ &= 8 \times 8192 \times 32 \times 1024 \\ &= 65,536 \times 32,768 \\ &= 2,147,483,648 \text{ parameters}\end{aligned}$$

Step 2: Multiply by Byte Precision (2 Bytes for bf16)

$$\begin{aligned}\text{Total Bytes} &= 2,147,483,648 \times 2 \text{ bytes} \\ &= 4,294,967,296 \text{ bytes}\end{aligned}$$

Step 3: Convert to Gigabytes

$$\begin{aligned}\text{VRAM}_{\text{KVCache}} &= \frac{4,294,967,296}{1024^3} \\ &= 4.0 \text{ GB}\end{aligned}$$

- **Conclusion:** Just hosting the KV Cache for 4 users at 8k context length consumes **4.0 GB** of VRAM, entirely separate from the 16.0 GB of VRAM required to store the static model weights!

Tensor & Shape Tracking

For a single generation step t, where input is `x_new` of shape `[B, 1, d]`:

- **Query vector (q_t)**: `[B, H, 1, d_k]`
- **New Key/Value vectors (k_t, v_t)**: `[B, G, 1, d_k]`
- **Stored Cache Tensors ($K_{\text{cache}}, V_{\text{cache}}$)**: `[B, G, t-1, d_k]`
- **Appended Cache Tensors ($K_{\text{cache\_new}}, V_{\text{cache\_new}}$)**: `[B, G, t, d_k]`
- **Attention weight output (QK^T)**: `[B, H, 1, t]` (representing query attending to all t past positions).

---

3. Implementation & Reference Code

Below is a self-contained PyTorch simulation of an autoregressive generation step using a KV cache helper class.

```python
import torch
import torch.nn as nn

class KVCacheManager:
    def __init__(self, num_layers: int, num_heads: int, head_dim: int):
        self.num_layers = num_layers
        self.num_heads = num_heads
        self.head_dim = head_dim
        # Caches are stored as lists of tuples: (k_cache, v_cache)
        self.k_caches = [None] * num_layers
        self.v_caches = [None] * num_layers

     def update(self, layer_idx: int, k_new: torch.Tensor, v_new: torch.Tensor) ->
tuple[torch.Tensor, torch.Tensor]:
        # k_new, v_new shapes: [B, h, 1, d_k]
        if self.k_caches[layer_idx] is None:
            # First token (prefill)
            self.k_caches[layer_idx] = k_new
            self.v_caches[layer_idx] = v_new
        else:
            # Concatenate along sequence dimension (dim=2)
              self.k_caches[layer_idx] = torch.cat([self.k_caches[layer_idx], k_new],
dim=2)
              self.v_caches[layer_idx] = torch.cat([self.v_caches[layer_idx], v_new],
dim=2)

        return self.k_caches[layer_idx], self.v_caches[layer_idx]

    def reset(self):
        self.k_caches = [None] * self.num_layers
        self.v_caches = [None] * self.num_layers

# Verification block simulating 3 decode steps
if __name__ == "__main__":
    B, h, d_k = 2, 8, 64
    n_layers = 12

    cache_manager = KVCacheManager(num_layers=n_layers, num_heads=h, head_dim=d_k)

    # Simulate step 1: Prefill sequence length L=5
    print("--- STEP 1: PREFILL PHASE ---")
    k_prefill = torch.randn(B, h, 5, d_k)
    v_prefill = torch.randn(B, h, 5, d_k)
         k_cached,  v_cached  =  cache_manager.update(layer_idx=0,  k_new=k_prefill,
v_new=v_prefill)
     print("Cached Keys shape after prefill:", k_cached.shape) # Expected: [2, 8, 5,
64]

    # Simulate step 2: Generate first token (new sequence length L=1)
    print("\n--- STEP 2: DECODING STEP 1 ---")
    k_new_1 = torch.randn(B, h, 1, d_k)
    v_new_1 = torch.randn(B, h, 1, d_k)
          k_cached,  v_cached  =  cache_manager.update(layer_idx=0,  k_new=k_new_1,
```

```
v_new=v_new_1)
    print("Cached Keys shape after decode 1:", k_cached.shape) # Expected: [2, 8, 6,
64]

    assert k_cached.shape == (B, h, 6, d_k), "Cache concatenation shape mismatch!"
    print("KV Cache Manager successfully concatenated cache states!")
```

---

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Eliminating the $O(L^2)$ redundant computations of calculating past token representations during token-by-token decoding.
- **Why Introduced over Legacy Approaches:** Autoregressive models must attend to historical contexts. KV Caching trade computation flops for memory bytes, enabling fast generations at the cost of VRAM footprint.
- **Key Failure Modes & Limitations:** Dynamic allocations on memory heaps create massive page fragmentations. This forces maximum batch size reductions to avoid OOMs, even when average VRAM usage is low.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Projection flops scale as $O(B \cdot 1 \cdot d^2)$ per step. Attention dot product scales as $O(B \cdot H \cdot 1 \cdot t \cdot d_k)$.
- **Space/Memory Footprint:** Scaled VRAM allocation of $O(2 \cdot B \cdot L \cdot n_{\text{layers}} \cdot n_{\text{heads\_kv}} \cdot d_{\text{head}} \cdot \text{BytesPerParam})$.
- **Primary Bottleneck Type:** Memory-bandwidth-bound (HBM to SRAM transfers).
- **Variable Legend:** B = Batch Size, L = Sequence Length, t = Current token step index, n_{layers} = Layer count.

3. Production & Scalability

- **Deployment Considerations:** Under long context windows, KV Caches can exceed the weights parameters size. Dynamic memory management tools like PagedAttention (Module 07) must be deployed to avoid static pre-allocation waste.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

Question: Contrast the computing profile of the prefill phase vs. the decoding phase.

The prefill phase processes the entire prompt sequence L in parallel, making it highly compute-bound (dominated by matrix-matrix multiplications). The decoding phase processes only one token at a time, making it memory-bandwidth bound (dominated by loading model weights and the KV Cache from HBM into SRAM caches).

Question: Calculate the KV Cache VRAM saving when migrating from MHA to GQA on Llama-3-70B (80 layers, 64 Query heads, 8 KV heads, head dim 128, batch 8, sequence length 4096, fp16).

Under MHA (64 KV heads):

$$\text{VRAM}_{\text{MHA}} = 4 \times 8 \times 4096 \times 80 \times 64 \times 128 \times 2 = 171.79 \text{ GB}$$

Under GQA (8 KV heads):

$$\text{VRAM}_{\text{GQA}} = 4 \times 8 \times 4096 \times 80 \times 8 \times 128 \times 2 = 21.47 \text{ GB}$$

GQA saves 150.32 GB of VRAM, making long-context 70B inference possible on a single multi-GPU node.

Module 07: GPU Memory Bounds & Inference Optimizations (FlashAttention & PagedAttention)

1. Introduction & Intuition

The Core Bottleneck

GPUs possess massive computational power (thousands of TFLOPS), but accessing memory is slow. There are two primary types of memory on a GPU:

1. **High Bandwidth Memory (HBM)**: Large global storage (e.g. 80GB on H100) where model weights and KV caches are stored. Accessing HBM is slow.
2. **SRAM**: Fast, local cache memory (a few megabytes) located directly on-chip near the GPU processors. Accessing SRAM is fast.

The core bottleneck in self-attention is the intermediate attention weight matrix $A = \text{softmax}(QK^T/\sqrt{d_k})$. This matrix has size $L \times L$. For a sequence length $L = 64\text{k}$, this matrix occupies 8 GB of memory for a single head! In standard PyTorch, this matrix is computed, written to HBM, read back from HBM to apply softmax, and read again to multiply by V. This roundtrip read/write cycle of $O(L^2)$ intermediate values to slow HBM is a major memory-bandwidth bottleneck, slowing execution down to a crawl.

Furthermore, when serving models, the KV cache of multiple users must be stored. Standard deep learning frameworks allocate KV cache memory statically as contiguous arrays. Because sequence lengths are dynamic, frameworks pre-allocate memory for the maximum possible length. This results in **memory fragmentation and waste (up to 60-80%)**, limiting the number of requests a GPU can serve concurrently.

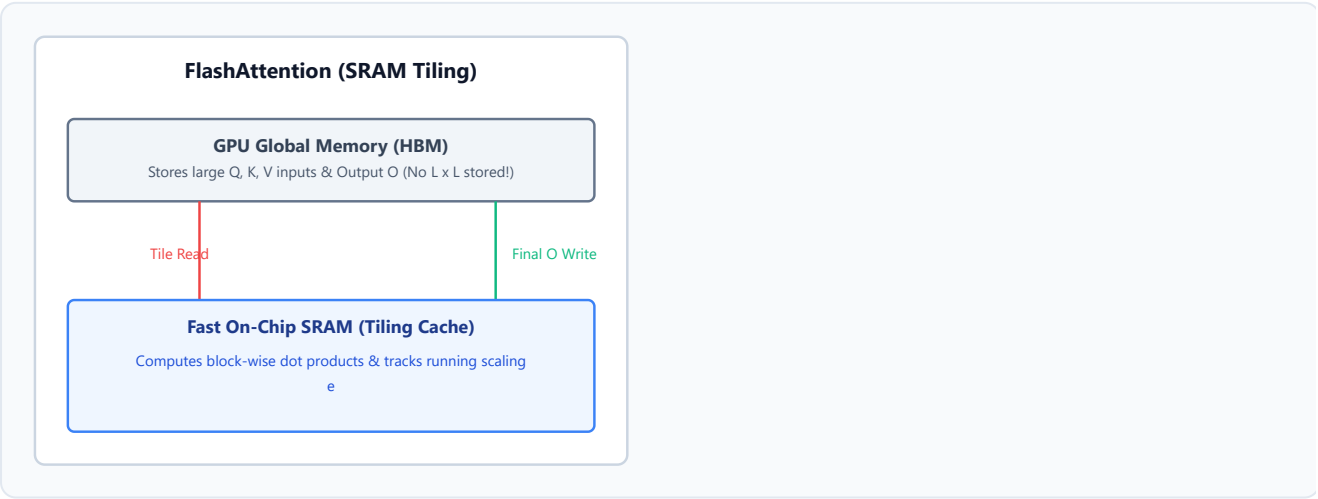
High-Level Intuition

- **FlashAttention**: Bypasses the HBM bottleneck by using **tiling**. Instead of loading the entire Query, Key, and Value matrices to HBM to compute the $L \times L$ grid, it loads small blocks (tiles) of Q, K, and V into fast on-chip **SRAM**. It computes attention locally on these tiles, updates the output, and writes the output back to HBM. By using **online softmax** tracking, it normalizes the values incrementally. The intermediate $L \times L$ attention matrix is never written to HBM, reducing memory accesses from quadratic $O(L^2)$ to linear $O(L)$.
- **PagedAttention**: Solves the memory allocation bottleneck by copying the concept of **virtual memory paging** from operating systems. Instead of allocating a contiguous block of VRAM for

each request's KV Cache, PagedAttention splits the KV Cache into small, fixed-size **pages** (representing e.g. 16 tokens). These pages are mapped to non-contiguous locations in VRAM via a lookup table. When a new token is generated, it is written to the current physical page. If the page is full, a new physical block is allocated from a shared pool. This eliminates fragmentation, enabling up to 4x higher throughput.

FlashAttention Tiling & PagedAttention Virtual Mapping

Below is an inline SVG illustrating the architectural layout of FlashAttention and PagedAttention:



x - m statistics updated incrementally PagedAttention (Virtual Mapping) Logical Cache Block 0 (T0-15) Block 1 (T16-31) Page Table Mapping Physical VRAM VRAM Block 14 VRAM Block 89 Scatter Allocated

---

2. Core Concepts & Mathematical Formulation

Mathematical Formulations

1. Roofline Model Operational Intensity

$$\text{Operational Intensity} = \frac{\text{FLOPs}}{\text{Memory Access (Bytes)}}$$

- **Purpose & High-level Intuition:** Measures whether an execution phase is bound by memory latency or processor speeds.
  - **Compute-Bound**: Operational intensity is high. The GPU spends its time performing arithmetic. High GPU utilization is achieved.
  - **Memory-Bandwidth-Bound**: Operational intensity is low. The arithmetic units are idle, waiting for data to load from HBM. GPU utilization drops.

2. Online Softmax Normalization

To prevent numerical overflow when computing softmax on hardware, we subtract the maximum value:

$$\text{softmax}(x)_i = \frac{e^{x_i - m}}{\sum_j e^{x_j - m}}, \quad \text{where } m = \max_j x_j$$

In FlashAttention, this is computed incrementally for tiles. If we have local block maximum $m^{(1)}$ and sum $s^{(1)}$, and receive new block with maximum $m^{(2)}$ and sum $s^{(2)}$, we update the running maximum and sum:

$$m^{\text{new}} = \max(m^{(1)}, m^{(2)})$$

$$s^{\text{new}} = s^{(1)} e^{m^{(1)} - m^{\text{new}}} + s^{(2)} e^{m^{(2)} - m^{\text{new}}}$$

Hand Calculations: Decoding Operational Intensity

Let's calculate the operational intensity of generating a single token in the decoding phase.

Parameters

- Model Parameters (N) $= 7,000,000,000$ (7B parameters).
- Precision: float16 (2 bytes per parameter).
    - **FLOPs required**: $2 \times N$ calculations (matrix-vector multiplication of input token with all weights).

$$\begin{aligned} \text{FLOPs} &= 2 \times N \\ &= 2 \times 7 \times 10^9 \\ &= 14 \times 10^9 \text{ FLOPs } (14 \text{ GFLOPs}) \end{aligned}$$

    - **Memory bytes read**: Load all parameters from HBM to registers.

$$\begin{aligned} \text{Bytes Read} &= N \times 2 \text{ bytes} \\ &= 7 \times 10^9 \times 2 \text{ bytes} \\ &= 14 \times 10^9 \text{ Bytes } (14 \text{ GB}) \end{aligned}$$

Calculate Operational Intensity

$$\begin{aligned}\text{Operational Intensity} &= \frac{\text{FLOPs}}{\text{Bytes Read}} \\ &= \frac{14 \times 10^9 \text{ FLOPs}}{14 \times 10^9 \text{ Bytes}} \\ &= 1.0 \text{ FLOP/Byte}\end{aligned}$$

- **Conclusion:** The operational intensity is exactly **1.0 FLOP/Byte**.
  An NVIDIA A100 GPU has a roofline threshold of around **150 FLOPs/Byte**. Since our intensity (1.0) is way below the threshold, the decoding phase is extremely **memory-bandwidth bound**. The GPU processors spend 99% of their time idle, waiting for weights to load from global memory.

Tensor & Shape Tracking

- **Logical KV Cache Block**: `[Block_Size, G, d_k]` (where `Block_Size` is typically 16 tokens).
- **FlashAttention Block (Tile)**:
  - Query Tile (Q_i): `[B_r, d_k]` (where B_r is row block size, e.g. 64).
  - Key Tile (K_j): `[B_c, d_k]` (where B_c is column block size, e.g. 64).
  - Value Tile (V_j): `[B_c, d_k]`.

---

3. Implementation & Reference Code

Below is a Python simulation of the incremental online softmax math used during FlashAttention tiling.

```
import numpy as np

def compute_standard_softmax(x):
    # Standard softmax subtraction to avoid numerical overflow
    m = np.max(x)
    exp_x = np.exp(x - m)
    return exp_x / np.sum(exp_x), m, np.sum(exp_x)

def simulate_online_softmax():
    # Simulate an attention row split into two blocks
    np.random.seed(42)
    x = np.random.randn(8)

    block1 = x[:4]
    block2 = x[4:]

    # 1. Process Block 1
    m1 = np.max(block1)
    s1 = np.sum(np.exp(block1 - m1))
```

```
    o1 = np.exp(block1 - m1) / s1

    # 2. Process Block 2
    m2 = np.max(block2)
    s2 = np.sum(np.exp(block2 - m2))
    o2 = np.exp(block2 - m2) / s2

    # 3. Apply FlashAttention Online Softmax Update Formula
    m_new = max(m1, m2)
    s_new = s1 * np.exp(m1 - m_new) + s2 * np.exp(m2 - m_new)

    # Normalize output representations to the new scale
    o1_scaled = o1 * (s1 * np.exp(m1 - m_new) / s_new)
    o2_scaled = o2 * (s2 * np.exp(m2 - m_new) / s_new)

    final_online = np.concatenate([o1_scaled, o2_scaled])

    # Compare with standard softmax
    final_standard, _, _ = compute_standard_softmax(x)

    print("Standard Softmax Output:", final_standard)
    print("Online Softmax Output  :", final_online)
    assert np.allclose(final_standard, final_online), "Outputs do not match!"
    print("Online softmax arithmetic successfully verified!")

if __name__ == "__main__":
    simulate_online_softmax()
```

---

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** The memory-bandwidth wall of writing/reading $L \times L$ attention matrices, and static KV cache allocation fragmentation.
- **Why Introduced over Legacy Approaches:** PyTorch's native attention creates quadratic HBM roundtrips. FlashAttention bypasses this by doing block-wise SRAM reductions. PagedAttention replaces contiguous arrays with dynamic page mappings to maximize serving batch sizes.
- **Key Failure Modes & Limitations:** PagedAttention adds lookup overhead via the page table, slightly increasing routing latency.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Both standard attention and FlashAttention scale as $O(L^2 \cdot d)$ computation FLOPs.
- **Space/Memory Footprint:** Standard attention requires $O(L^2 \cdot h)$ HBM storage for intermediate activations. FlashAttention requires $O(L \cdot d)$ linear storage.

- **Primary Bottleneck Type:** FlashAttention is Compute-bound (or memory-bandwidth bound depending on context length); PagedAttention overhead is Memory-bandwidth bound.
- **Variable Legend:** L = Sequence Length, d = Hidden Model Dimension, h = Head Count.

3. Production & Scalability

- **Deployment Considerations:** FlashAttention is implemented as custom CUDA kernels. It requires compilation and direct hardware support (e.g. FP16/BF16 tensor cores on Ampere/Hopper).

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

> **Question: Why does FlashAttention perform *more* FLOPs than standard attention during backpropagation, but runs significantly faster?**
>
> In the backward pass, FlashAttention does not load the $L \times L$ attention matrix from HBM (since it was never stored). Instead, it recomputes the attention matrix tiles on-the-fly in SRAM. This requires performing extra FLOP calculations. However, because HBM memory reads/writes are 10-100x slower than SRAM compute FLOPs, recomputing is much faster than loading from memory.

> **Question: Explain how PagedAttention enables "Copy-on-Write" for parallel decoding.**
>
> When generating multiple completions for a prompt (e.g. beam search or system branching), the initial prompt's KV cache is shared. Instead of copying the prompt's KV cache for each path, PagedAttention points all virtual tables to the same physical pages (with read-only locks). When one path generates a new token, only its target page is copied and modified (Copy-on-Write). This saves massive amounts of VRAM.

Module 08: Pre-training: Architecture Styles, Objectives & Data Engineering

1. Introduction & Intuition

The Core Bottleneck

Training state-of-the-art LLMs consumes millions of dollars in compute budgets. The pre-training phase exposes a model to trillions of tokens to learn general representations and grammar. Historically, researchers scaled models by simply increasing the parameter size N while keeping the training token count D relatively small.

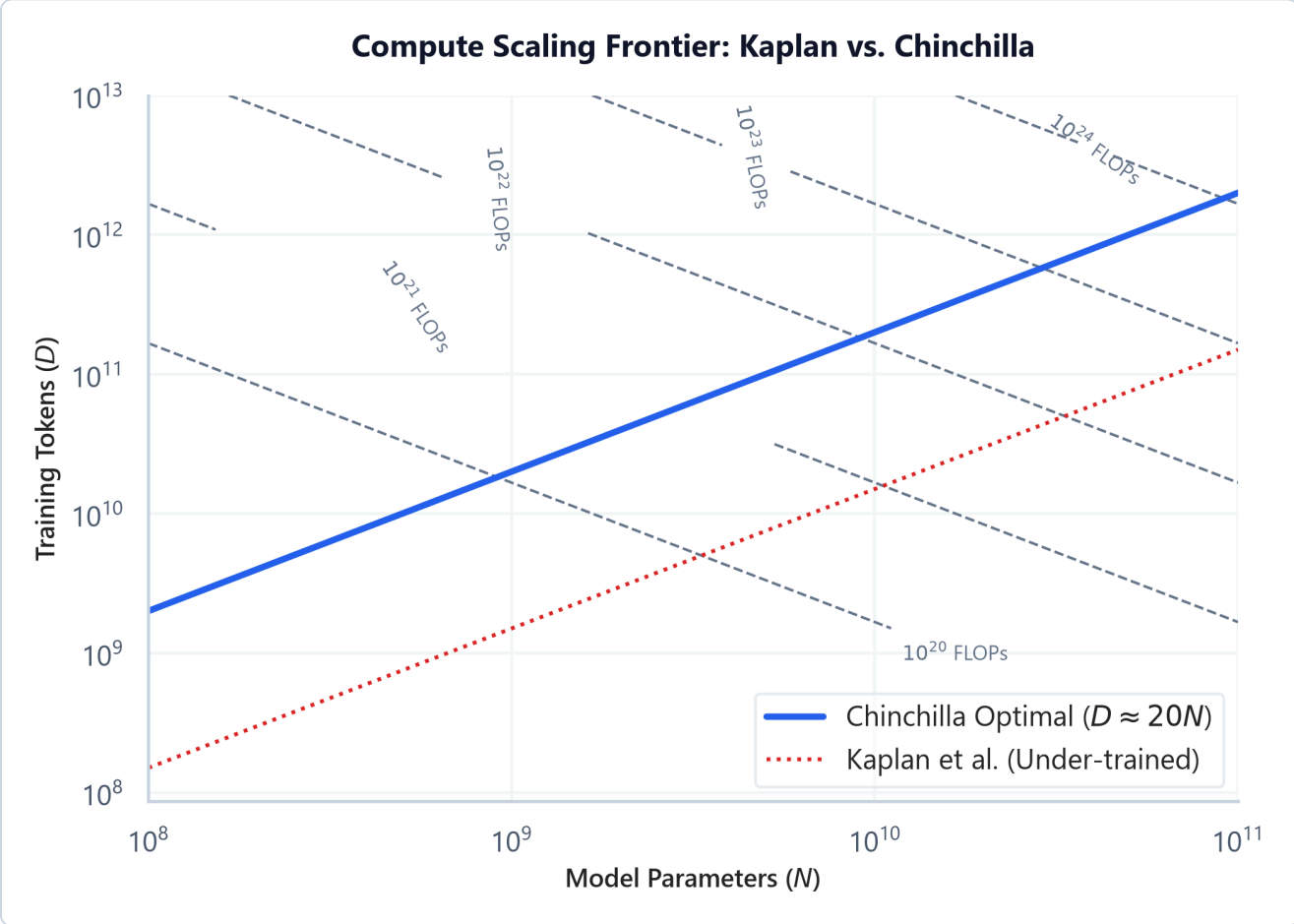
This led to a major bottleneck: **Compute Inefficiency**. As discovered by Kaplan et al. and corrected by Chinchilla (Hoffmann et al.), many early models (like GPT-3 175B) were severely under-trained because they did not have enough tokens relative to their size. The bottleneck is finding the optimal trade-off between model parameters N and dataset size D to maximize accuracy under a fixed training compute budget.

High-Level Intuition

- **Architecture Styles**:
  - **Encoder-Only (BERT)**: Uses masked language modeling (predicts missing tokens in the middle). Ideal for extraction, classification, and embeddings.
  - **Decoder-Only (GPT, Llama)**: Uses causal language modeling (predicts the next token from left-to-right). Ideal for generative tasks, reasoning, and conversational agents.
  - **Encoder-Decoder (T5, BART)**: Combines both. Ideal for translation, summarization, and sequence mapping.
- **Scaling Laws**:
  - **Kaplan's Law**: Claimed that parameter size N should grow much faster than training tokens D as compute increases.
  - **Chinchilla's Law**: Proved that parameters and tokens should scale in a **1:1 proportion**. For every doubling of compute, we should increase the model size by $1.4\times$ and the token count by $1.4\times$. This means a model should be trained on approximately $20 \text{ tokens per parameter}$ to be compute-optimal.

Compute Scaling Frontier: Kaplan vs. Chinchilla

Below is the pre-generated scaling chart demonstrating optimal allocations under compute constraints:



- **Plot Interpretation:** The dotted contour curves represent lines of constant training compute budgets (in FLOPs). The blue line shows the Chinchilla optimal path ($D \approx 20N$), demonstrating that as compute budgets scale up, model size and token count should be scaled in equal proportions. The red dotted line represents Kaplan's scaling strategy, which results in over-parameterized and under-trained models (low token-to-parameter ratio) that underperform on downstream benchmarks relative to their size.

---

2. Core Concepts & Mathematical Formulation

Mathematical Formulations

1. Pre-training Compute Cost Estimator (FLOPs)

For decoder-only models, the computational budget C to train a model is:

$$C \approx 6ND$$

- **Purpose & High-level Intuition:** This formula estimates the total floating-point operations (FLOPs) required to pre-train a model of size N (parameters) on a dataset of size D (tokens).
  - **Forward Pass**: Requires $2ND$ FLOPs (2 floating-point operations per parameter per token: one multiply, one add).
  - **Backward Pass**: Requires $4ND$ FLOPs (gradients are calculated and accumulated, which takes twice the operations of the forward pass).
  - **Total**: $2ND + 4ND = 6ND$ FLOPs.

2. Chinchilla Optimal Scaling Equation

Under a fixed compute budget C:

$$C = 6ND \implies N \propto \sqrt{C}, \quad D \propto \sqrt{C}$$

- **Purpose & High-level Intuition:** Restates that model size and training tokens should scale equally ($N \approx 20D$ tokens, or more precisely $D \approx 20N$).

Hand Calculations: Chinchilla Optimal Allocation

Let's calculate the compute-optimal model size N and token count D for a training budget of $C = 6 \times 10^{23}$ FLOPs.

Step 1: Set up the Chinchilla ratio rule of thumb

Assume the optimal dataset size D is proportional to model parameters N by a factor of 20:

$$D = 20N$$

Step 2: Substitute into the compute budget equation

$$\begin{aligned} C &= 6ND \\ 6 \times 10^{23} &= 6 \times N \times (20N) \\ 6 \times 10^{23} &= 120N^2 \\ N^2 &= \frac{6 \times 10^{23}}{120} \\ N^2 &= 5 \times 10^{21} \\ N &= \sqrt{5 \times 10^{21}} \\ N &\approx 7.071 \times 10^{10} \text{ parameters } (70\text{B}) \end{aligned}$$

Step 3: Compute optimal token count

$$\begin{aligned} D &= 20N \\ &= 20 \times 7.071 \times 10^{10} \\ &= 1.414 \times 10^{12} \text{ tokens } (1.4\text{T}) \end{aligned}$$

- **Conclusion:** For a training budget of 6×10^{23} FLOPs, the optimal model size is **70 Billion parameters** trained on **1.4 Trillion tokens**. Training a larger model (e.g. 100B) on fewer tokens, or a smaller model (e.g. 30B) on more tokens, would result in worse performance for this budget.

Tensor & Shape Tracking

- **Vocabulary Output Projection**: `[B * L, V]`
- **Cross-Entropy Loss Input**: `[B * L, V]` against targets `[B * L]` (shifted by 1 token for causal prediction).
- **Weights shape**: `[N]` total variables.

---

3. Implementation & Reference Code

Below is a Python simulation tracking Chinchilla compute allocations and training FLOP counts.

```
def estimate_training_time_and_compute(params_billions: float, tokens_billions: float, gpu_tflops: float, hardware_utilization: float = 0.45):
    # Convert inputs to raw scales
    N = params_billions * 1e9
    D = tokens_billions * 1e9

    # 1. Compute total training FLOPs
    flops = 6 * N * D

    # 2. Compute effective hardware throughput per GPU per second
    effective_tflops = gpu_tflops * hardware_utilization # Account for overhead (MFU)
    flops_per_gpu_day = effective_tflops * 1e12 * 60 * 60 * 24

    # Let's assume a cluster of 512 GPUs
    num_gpus = 512
    total_flops_per_day = flops_per_gpu_day * num_gpus

    training_days = flops / total_flops_per_day

    print(f"Model Parameters: {params_billions}B")
    print(f"Dataset Size    : {tokens_billions}B tokens")
    print(f"Total FLOPs     : {flops:.2e} FLOPs")
    print(f"Training Time on {num_gpus} GPUs: {training_days:.2f} days")
    return flops, training_days
```

```
if __name__ == "__main__":
    # Estimate for a 7B model trained on 2 Trillion tokens (Llama-2 style)
    # Using A100 GPU specs: 312 TFLOPS bfloat16 tensor cores
    estimate_training_time_and_compute(
        params_billions=7.0,
        tokens_billions=2000.0,
        gpu_tflops=312.0,
        hardware_utilization=0.45
    )
```

---

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** The computational waste of training models that are either too large (under-trained relative to parameter sizes) or too small (under-performing) for a given compute budget.
- **Why Introduced over Legacy Approaches:** Chinchilla proved that scaling datasets is just as critical as scaling weights, changing industry practices towards training smaller models longer (over-training) to reduce inference serving costs.
- **Key Failure Modes & Limitations:** Chinchilla optimization only minimizes training compute. In production, we often prefer to train a model *past* the Chinchilla limit (over-training) because the higher training cost is amortized by the lower inference cost of a smaller model.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Total pre-training training cost scales linearly as $O(6ND)$.
- **Space/Memory Footprint:** Parameter weights require $N \times \text{BytesPerParam}$. Optimization state VRAM scales as $16 \times N$ to $20 \times N$ bytes (using Adam optimizer).
- **Primary Bottleneck Type:** Compute-bound during matrix projections; Network/IO bound during distributed pipeline communication (AllReduce).
- **Variable Legend:** N = Parameter Count, D = Token Count.

3. Production & Scalability

- **Deployment Considerations:** Training models with tens of billions of parameters requires distributed systems techniques, including Tensor Parallelism (splitting layers across GPUs) and Pipeline Parallelism (splitting sequential blocks across cards).

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

Question: Why are modern models like Llama 3 trained *beyond* the Chinchilla optimal limit (e.g. Llama 3 8B was trained on 15T tokens, a ratio of 1800+ tokens/param instead of 20)?

Chinchilla optimal scaling only minimizes training compute cost. However, in commercial settings, a model is trained once but served billions of times at inference. Training a smaller 8B model longer costs more upfront but keeps inference VRAM and latency footprints low. A 70B model trained on 1.4T tokens might match the 8B model's quality, but serving the 70B model would be 8x more expensive at inference.

Question: Derive the memory overhead of the Adam optimizer during training.

For N parameters in mixed-precision training (FP16/BF16): Model parameters: $2N$ bytes. Model gradients: $2N$ bytes. Master weights copy (FP32): $4N$ bytes (for stable gradient updates). Adam momentum state (FP32): $4N$ bytes. Adam variance state (FP32): $4N$ bytes. Total optimizer overhead: $16N$ bytes. Storing a 7B model during training requires at least $20 \times 7 = 140\ \text{GB}$ of VRAM just for training state variables, before accounting for activations.

Module 09: Architectural Dissection of Frontier Models, MoE, & Deep Thinking Models

1. Introduction & Intuition

The Core Bottleneck

As dense LLMs scale up, the compute cost to train and run them grows. For a dense 100B model, every single token generation requires running the entire 100B parameter network. This creates a major bottleneck: **Inference Compute Footprint**.

To scale capacity without scaling compute, researchers designed **Mixture of Experts (MoE)**. Instead of using a single large FFN layer, MoE replaces the FFN with multiple parallel **experts** (smaller FFN blocks). A router network selects the top-k experts for each token. Only the parameters of the selected experts are activated. However, MoE creates a memory bottleneck: all experts must be kept in VRAM, which requires a large memory footprint.

Recently, a new bottleneck emerged: **System Limits on Direct Output Generation**. Standard models generate responses in a single forward-pass sweep, which limits their performance on complex mathematical and logical reasoning tasks. To solve this, **Deep Thinking/Reasoning Models** (e.g. OpenAI o1/o3, DeepSeek-R1) shift compute from training to **inference** (Test-Time Compute). By generating long intermediate chains of thoughts, verifying steps, and searching reasoning trees, they resolve complex tasks at the cost of higher generation latency.

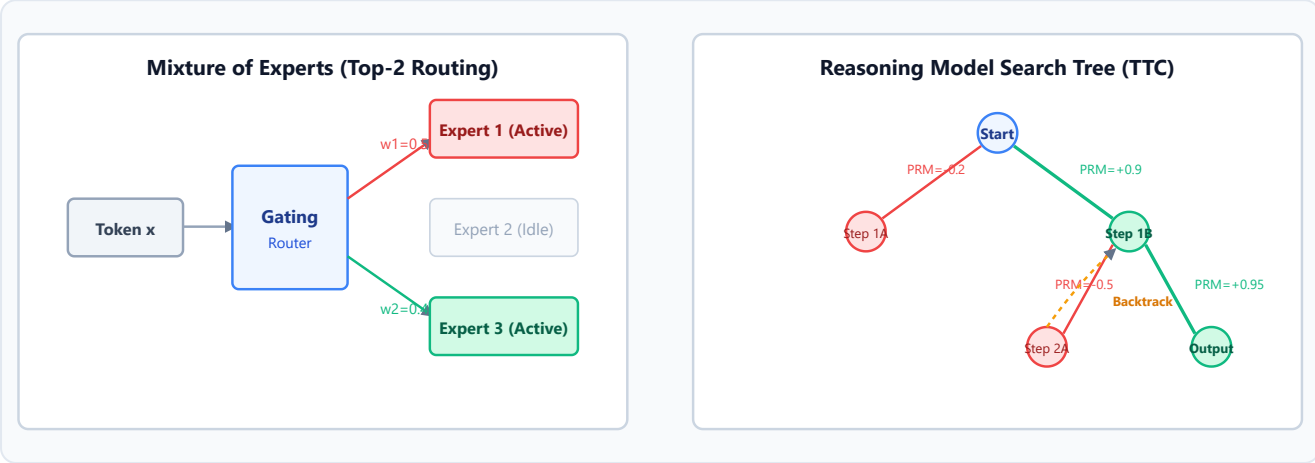
High-Level Intuition

- **Frontier Configuration Trends**: Modern models (Llama 3, Mistral) have standard configurations: Pre-LN RMSNorm, RoPE positional embeddings, SwiGLU activation functions, Grouped-Query Attention (GQA), and large vocabulary sizes.
- **Mixture of Experts (MoE)**: Think of a general contractor routing tasks to specialists. Instead of one generalist FFN doing all the work, the router sends math tokens to the mathematician expert, code tokens to the programmer expert, and translation tokens to the linguist expert. The output is a weighted sum of their work.
- **Deep Thinking Models & Test-Time Compute (TTC)**: Think of solving a hard riddle. Instead of blurting out the first answer that comes to mind (standard LLMs), you pause, write out a list of sub-problems, verify each step, backtrack if you hit a contradiction, and only state the final

solution once you are confident. This is implemented via Reinforcement Learning over search trees.

MoE Router Routing & Reasoning Search Trees

Below is an inline SVG demonstrating MoE routing and Reasoning Search paths:



---

2. Core Concepts & Mathematical Formulation

Mathematical Formulations

1. MoE Router Softmax

To select the top-K experts for a token x:

$$G(x)_i = \text{Softmax}(\text{KeepTopK}(H(x), K))_i$$

$$\text{where KeepTopK}(v, K)_i = \begin{cases} v_i & \text{if } v_i \text{ is in top } K \text{ values} \\ -\infty & \text{otherwise} \end{cases}$$

- **Purpose & High-level Intuition:** The router maps token representation x to a score vector $H(x) = x \cdot W_{\text{gate}}$. To enforce sparsity, we zero out all scores except the top-K experts by setting them to $-\infty$. The softmax converts the remaining scores into routing weights that sum to 1. This routes the token to a subset of experts, keeping compute constant while scaling parameter capacity.

Hand Calculations: Top-2 Router Gating

Let's compute the routing weight vector for a token x mapped to $N_{\text{experts}} = 4$ experts.
Assume the router raw outputs are:

$$H(x) = \begin{pmatrix} 0.5 & 1.2 & 0.1 & 1.0 \end{pmatrix}$$

Let the number of active experts be $K = 2$.

Step 1: Apply KeepTopK Filter

Find the top 2 values in $H(x)$:

- Highest value: 1.2 at index 1.
- Second highest value: 1.0 at index 3.
  Zero out all other indices by setting them to $-\infty$:

$$\bar{H}(x) = \begin{pmatrix} -\infty & 1.2 & -\infty & 1.0 \end{pmatrix}$$

Step 2: Compute Softmax

$$\begin{aligned} \text{Sum} &= e^{1.2} + e^{1.0} \\ &\approx 3.3201 + 2.7183 \\ &= 6.0384 \end{aligned}$$

- **Gating weight for Expert 1**:

$$\begin{aligned} G(x)_1 &= \frac{e^{1.2}}{6.0384} \\ &= \frac{3.3201}{6.0384} \\ &\approx 0.5498 \end{aligned}$$

- **Gating weight for Expert 3**:

$$\begin{aligned} G(x)_3 &= \frac{e^{1.0}}{6.0384} \\ &= \frac{2.7183}{6.0384} \\ &\approx 0.4502 \end{aligned}$$

- **Gating weights for Expert 0 and 2**:

$$G(x)_0 = 0.0, \quad G(x)_2 = 0.0$$

The final routing vector is:

$$G(x) = \begin{pmatrix} 0.0 & 0.5498 & 0.0 & 0.4502 \end{pmatrix}$$

- **Conclusion:** Token x is routed to Expert 1 (with 55% weight) and Expert 3 (with 45% weight). The parameters of Expert 0 and Expert 2 are not loaded or executed for this token.

Tensor & Shape Tracking

- **Gating projection matrix (W_{gate})**: `[d_model, N_experts]`
- **Expert input (token representation)**: `[B * L, d_model]`
- **Routed Expert outputs (each)**: `[N_tokens_assigned, d_model]`
- **Assembled FFN output**: `[B, L, d_model]`

---

3. Implementation & Reference Code

Below is a self-contained PyTorch simulation of an MoE routing block routing tokens to parallel expert networks.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class Expert(nn.Module):
    def __init__(self, d_model: int, d_ffn: int):
        super().__init__()
        self.w1 = nn.Linear(d_model, d_ffn, bias=False)
        self.w2 = nn.Linear(d_ffn, d_model, bias=False)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # Standard ReLU FFN expert
        return self.w2(F.relu(self.w1(x)))

class SparseMoELayer(nn.Module):
    def __init__(self, d_model: int, d_ffn: int, num_experts: int, top_k: int):
        super().__init__()
        self.num_experts = num_experts
        self.top_k = top_k

        # Router projection
        self.router = nn.Linear(d_model, num_experts, bias=False)
        # Collection of experts
                self.experts = nn.ModuleList([Expert(d_model, d_ffn) for _ in
range(num_experts)])

    def forward(self, x: torch.Tensor) -> torch.Tensor:
```

```
        # x shape: [B, L, d_model]
        B, L, d = x.shape
        flat_x = x.view(-1, d) # [B * L, d_model]

        # 1. Compute router scores
        logits = self.router(flat_x) # [B * L, num_experts]

        # 2. Get top-k scores and indices
        scores, indices = torch.topk(logits, self.top_k, dim=-1) # [B * L, top_k]

        # 3. Softmax over the top-k experts
        weights = F.softmax(scores, dim=-1) # [B * L, top_k]

        # 4. Route and execute tokens block-wise
        output = torch.zeros_like(flat_x)

        # We loop through experts and gather tokens assigned to each
        for exp_idx in range(self.num_experts):
            # Mask identifying which tokens route to this expert
            token_mask = (indices == exp_idx)
            if not token_mask.any():
                continue

            # Get flat indices of assigned tokens
            token_indices, k_indices = torch.where(token_mask)

            # Extract inputs and execute expert
            inputs = flat_x[token_indices]
            expert_outputs = self.experts[exp_idx](inputs)

            # Weight outputs and accumulate
            weights_extracted = weights[token_indices, k_indices].unsqueeze(-1)
            output[token_indices] += weights_extracted * expert_outputs

        return output.view(B, L, d)

# Verification block
if __name__ == "__main__":
    B, L, d = 2, 8, 16
    x = torch.randn(B, L, d)

    moe = SparseMoELayer(d_model=d, d_ffn=32, num_experts=4, top_k=2)
    out = moe(x)

    print("Input shape:", x.shape)
    print("MoE Output shape:", out.shape)
    assert out.shape == x.shape, "Shape mismatch!"
    print("Sparse MoE Layer successfully executed and verified routing loops!")
```

---

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** The quadratic computational cost scaling of scaling dense networks. MoE scales parameter capacity while keeping activation FLOPs constant.
- **Why Introduced over Legacy Approaches:** MoE enables models to learn specialized tasks (e.g. math, coding, languages) within separate parameters blocks, improving downstream accuracy without increasing inference compute requirements.
- **Key Failure Modes & Limitations:** Routing collapses: tokens get routed to a subset of experts (e.g. only Expert 1 and 2), causing other experts to remain under-trained. This is mitigated using a **Load Balancing Loss** during training.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Linear projections scale as $O(2 \cdot B \cdot L \cdot d \cdot d_{\text{ffn}} \cdot K)$ where K is active experts (much lower than full N-expert evaluations).
- **Space/Memory Footprint:** Parameter footprint scales as $O(N_{\text{experts}} \cdot 2 \cdot d \cdot d_{\text{ffn}})$. All expert parameters must be loaded into VRAM, increasing memory requirements.
- **Primary Bottleneck Type:** Memory-routing and inter-GPU communication latency (All-to-All network exchanges).
- **Variable Legend:** N_{experts} = Total Expert count, K = Number of active routed experts.

3. Production & Scalability

- **Deployment Considerations:** Serving MoE requires massive VRAM. Model sharding is typically required: experts are split across multiple GPUs using Expert Parallelism.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

> **Question: Detail the difference between Process-Supervised Reward Models (PRMs) and Outcome-Supervised Reward Models (ORMs).**
>
> ORMs evaluate only the final output of a generation (assigning a reward based on whether the final answer is correct). PRMs evaluate every single intermediate step (thought block) of a reasoning trace. PRMs provide a dense feedback signal, which is critical for training deep thinking models because it rewards logical reasoning and penalizes correct answers arrived at via wrong steps.

> **Question: Explain what Test-Time Compute (TTC) scaling laws represent.**
>
> Traditional scaling laws focus on training compute (increasing parameters and training tokens). TTC scaling laws focus on inference compute: they state that for complex reasoning tasks, the performance of a model scales predictably as we allocate more compute budget at inference time (e.g. by expanding search paths, generating more thoughts, and verifying reasoning chains).

Module 10: Model Evaluation, Benchmark Metrics, & Alignment Evaluation

1. Introduction & Intuition

The Core Bottleneck

Evaluating LLMs is a challenging engineering task. Because LLMs generate free-form natural language, standard classification metrics (like Accuracy, F1-score) are not applicable.

In pre-training, we rely on **Perplexity (PPL)**, which measures how well a model predicts the next token. However, PPL only evaluates token probability distributions, not reasoning.

For reasoning, we rely on benchmarks like **MMLU** (multi-task knowledge), **GPQA** (PhD-level science reasoning), and **AIME/MATH** (mathematics).

The evaluation bottleneck is **Contamination and High Variance**:

- **Contamination**: Pre-training datasets are so large that benchmarks often leak into training sets, rendering results meaningless.
- **Stochasticity**: Tiny changes in prompts or parsing rules can cause wild swings in model outputs. Furthermore, aligning models using Reinforcement Learning requires a scoring mechanism. Deciding whether to evaluate only the final output (ORM) or each intermediate reasoning step (PRM) dictates how well a model can learn complex reasoning paths.

High-Level Intuition

- **Perplexity (PPL)**: Measures the model's uncertainty. Think of a multiple-choice exam. A model with low perplexity is confident and correct. A model with high perplexity is confused, guessing between many different options. Mathematically, it is the exponential of the cross-entropy loss.
- **Benchmarks**:
  - **MMLU**: Tests undergraduate-level general knowledge. Now saturated by top models.
  - **GPQA**: Graduate-level science questions designed to be extremely hard for AI and easy for PhD experts.
  - **AIME**: American Invitational Mathematics Examination problems, used to test high-level math reasoning.
- **ORM vs. PRM**:

- **Outcome-Supervised (ORM)**: Like a teacher who only grades the final answer on a math test. If you copy someone else's work but get the right number, you get a 100%. This can reward models that hallucinate incorrect reasoning chains but guess the right answer.
- **Process-Supervised (PRM)**: Like a teacher who grades your step-by-step working. If you make a mistake in step 3, you get penalized, even if you guess the final number. This directly rewards logical reasoning and helps train reasoning models.

Reward Modeling Evaluation Architectures

Below is a comparison of Outcome-Supervised (ORM) vs. Process-Supervised (PRM) reward boundaries:

Outcome-Supervised (ORM)

Evaluates **only the final result** of a reasoning chain.

Reasoning path: [Step 1] → [Step 2] → [Step 3] → **[Final Answer]** ← **Checked here**

Issue: Can reward correct answers arrived at via incorrect logic.

Process-Supervised (PRM)

Evaluates **every individual step** in the reasoning trace.

Reasoning path: [Step 1] ← **OK** → [Step 2] ← **OK** → [Step 3] ← **FAIL**

Benefit: Directly aligns model thoughts, helping RL optimize search trees.

---

2. Core Concepts & Mathematical Formulation

Mathematical Formulations

1. Perplexity (PPL)

$$\mathrm{PPL}(X) = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\ln P(x_i \mid x_{<i})\right)$$

- **Purpose & High-level Intuition:** Measures model uncertainty. In practice, we evaluate the cross-entropy loss $H(X) = -\frac{1}{N}\sum \ln P(x_i \mid x_{<i})$. Perplexity is simply the exponential of cross-entropy loss:

$$\mathrm{PPL}(X) = e^{H(X)}$$

A perplexity of V (vocab size) represents a model selecting uniformly at random. A perplexity of 1.0 represents a model predicting the sequence with absolute confidence and 100% accuracy.

Hand Calculations: Perplexity

Let's compute the perplexity for a short sequence of length $N = 2$.
Assume our model outputs the following conditional probabilities for the target tokens:

- Probability of token 1: $P(x_1) = 0.2231$ (log probability: $\ln(0.2231) \approx -1.5$).
- Probability of token 2: $P(x_2 \mid x_1) = 0.6065$ (log probability: $\ln(0.6065) \approx -0.5$).

Step 1: Compute Average Negative Log-Likelihood (Cross-Entropy Loss)

$$\begin{aligned}\text{Loss} &= -\frac{1}{2}\left(\ln P(x_1) + \ln P(x_2 \mid x_1)\right) \\ &= -\frac{1}{2}\left(-1.5 + (-0.5)\right) \\ &= -\frac{1}{2}\left(-2.0\right) \\ &= 1.0\end{aligned}$$

Step 2: Compute Perplexity

$$\begin{aligned}\text{PPL} &= e^{\text{Loss}} \\ &= e^{1.0} \\ &\approx 2.7183\end{aligned}$$

- **Conclusion:** The perplexity of the sequence is **2.72**. This means the model is, on average, as confused as if it were choosing between 2.72 options at each step.

Tensor & Shape Tracking

- **Logits Matrix**: `[B, L, V]`
- **Targets vector**: `[B, L]` (contains token IDs).
- **Loss vector (element-wise)**: `[B, L]`
- **Cross-entropy reduction output**: Scalar float `loss`.
- **PPL output**: Scalar float `ppl`.

---

3. Implementation & Reference Code

Below is a self-contained PyTorch implementation of Perplexity calculation from logits.

```python
import torch
import torch.nn as nn

def calculate_perplexity(logits: torch.Tensor, targets: torch.Tensor) -> float:
    # logits shape: [B, L, V]
    # targets shape: [B, L]
    B, L, V = logits.shape

    # 1. Reshape tensors for PyTorch CrossEntropyLoss
    # cross_entropy expects [B * L, V] inputs and [B * L] targets
    flat_logits = logits.view(-1, V)
    flat_targets = targets.view(-1)

    # 2. Compute average cross entropy loss
    # ignore_index is used to bypass padding tokens
    loss_fn = nn.CrossEntropyLoss(ignore_index=-100)
    loss = loss_fn(flat_logits, flat_targets)

    # 3. Take exponential of loss to obtain perplexity
    perplexity = torch.exp(loss)
    return loss.item(), perplexity.item()

# Verification block
if __name__ == "__main__":
    torch.manual_seed(42)
    B, L, V = 2, 4, 10

    # Mock logits and targets
    logits = torch.randn(B, L, V)
    targets = torch.tensor([
        [3, 5, 2, -100],  # Last token is padding
        [1, 9, 7, 0]
    ])

    loss, ppl = calculate_perplexity(logits, targets)
    print(f"Computed Cross-Entropy Loss: {loss:.4f}")
    print(f"Computed Perplexity: {ppl:.4f}")

    # Check bounds
    assert ppl > 1.0, "Perplexity cannot be less than 1.0!"
    print("Perplexity calculation successfully verified!")
```

---

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Establishing reliable evaluation metrics for free-form language generation, and providing reward signals for reinforcement learning.

- **Why Introduced over Legacy Approaches:** ROUGE/BLEU (overlap-based metrics) evaluate lexical similarities but fail to capture semantic accuracy or logical steps. PRMs grade reasoning steps directly, encouraging correct logic.
- **Key Failure Modes & Limitations:** Goodhart's Law: when a metric becomes a target, models optimize to score high on it (e.g. by exploiting benchmark multiple-choice layouts) without acquiring actual general intelligence.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Perplexity loss reduction runs in $O(B \cdot L \cdot V)$ operations.
- **Space/Memory Footprint:** Evaluations require hosting evaluation libraries, but add negligible runtime VRAM overhead.
- **Primary Bottleneck Type:** CPU-bound parsing routines; GPU-bound during evaluation inference loops.
- **Variable Legend:** B = Batch Size, L = Sequence Length, V = Vocabulary Size.

3. Production & Scalability

- **Deployment Considerations:** Online telemetry monitors model perplexity over time. A sudden jump in perplexity indicates a **data drift** (e.g. users inputting queries in new languages or formats that the model was not trained on).

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

> **Question: Why does a model's perplexity decrease as we train it on more tokens, but downstream task accuracies can sometimes plateau?**
>
> Perplexity measures how well the model predicts the token probability distribution. Initially, the model learns basic syntax, grammar, and sentence structures, which drastically reduces cross-entropy loss (and perplexity). However, downstream tasks often require high-level reasoning and specific knowledge, which depends on training past syntactic patterns.

> **Question: Explain how data contamination can be detected in LLM pre-training datasets.**
>
> Contamination can be detected by calculating model perplexity on the benchmark test questions. If a model shows an unnaturally low perplexity (e.g. close to 1.0) on a set of test questions, it suggests that these exact sequences were present in the training set (memorization). We also run substring matching and MinHash deduplication between datasets to verify cleanliness.